

Nordstrom Interview Preparation Guide

Ajay Venkatesh

Engineer 1 – Customer Experience Platform (Hybrid, Seattle WA)

Interviewer: Sangeeta Nori, Senior Manager Engineering, Core Digital Platforms

45-minute Teams video interview | June 2026

Contents

1	Part 1: Technical Questions with Detailed Answers	2
2	Part 2: Behavioral Questions with Detailed Answers	11
3	Part 3: Deep Technical Questions – CampusMesh (10)	18
4	Part 4: Deep Technical Questions – Amazon SDE Internship (10)	23
5	Part 5: Deep Technical Questions – NYU Projects (10)	28
6	Part 6: Docker and Kubernetes (5)	32
7	Quick Reference: Key Numbers to Remember	35
8	Day-of Checklist	35

Part 1: Technical Questions with Detailed Answers

Q1 – Microservices: Service Boundary Design (CampusMesh)

Question: Your CampusMesh backend runs multiple independent services – study groups, messaging, calls, notifications, profile management. How did you decide where to draw service boundaries, and what problems did splitting into microservices create that a monolith wouldn't have?

Answer: We drew service boundaries around **business domains**, not technical layers. Each service owns a coherent slice of user behavior: **study_group** handles creating, joining, and managing a study space; messaging owns real-time chat; calls owns WebRTC signaling; notifications owns push/SMS delivery. The rule was: if two features share a database table *and* are always deployed together, they stay in the same service. If they evolve at different rates or scale differently, they split.

Problems the split created: (1) **Distributed tracing** – a user joining a group triggers four services; correlating logs across Lambda invocations required propagating a correlation ID through every event. (2) **Data consistency** – MySQL handles relational data; DynamoDB handles high-throughput session and notification data; keeping them in sync when a service fails requires idempotent retry logic and dead-letter queues in SQS. (3) **Local development complexity** – running all services requires the Serverless Offline plugin with matching SSM parameters.

What a monolith would have avoided: one deployment, one transaction boundary, simple function calls. The split was worth it because services like recordings and notifications have very different load patterns – deploying them independently keeps blast radius small.

→ *Follow-up: How did services communicate – synchronous REST or async events?*

Both. Synchronous REST via API Gateway for user-initiated actions needing an immediate response (fetching group info, joining a group). Async events via **EventBridge** for cross-service side effects – when a user joins a group, an event fires to the notifications service. SQS buffers those events and provides retry guarantees. Rule: caller needs the result immediately ⇒ sync; caller doesn't care about timing ⇒ async.

→ *Follow-up: When one service fails, how did you prevent cascading failures?*

Three patterns: (1) **Dead-letter queues** on SQS – messages failing after 3 retries go to a DLQ for inspection. (2) **Timeouts** set to 10s on all API Gateway routes so a slow downstream can't hold connections indefinitely. (3) **Graceful degradation** – if notifications is down, the UI keeps working, just without real-time badge counts.

Q2 – Request Flow: API Gateway to MySQL

Question: Walk me through how a request flows in CampusMesh from the frontend hitting an API Gateway endpoint to MySQL returning data – every layer.

Answer: The full chain:

1. **Frontend** sends HTTPS with a JWT in **Authorization: Bearer** to the **CloudFront** URL. CloudFront serves static assets from the edge and routes dynamic API calls to API Gateway.
2. **API Gateway** matches the route (e.g., GET /study-groups/{id}) and invokes the Lambda. API Gateway does *not* validate the JWT.
3. **Lambda handler** (Node.js 20.x) runs a **JWT middleware** first: reads the token, verifies it with **JWT_SECRET** loaded from SSM Parameter Store at cold-start, extracts user ID and role. If invalid, returns 401 immediately.
4. **Handler logic** calls **Sequelize ORM** with authenticated user context. Sequelize builds a parameterized SQL query (no string interpolation – prevents SQL injection) and sends it to the MySQL connection pool. DB credentials are also loaded from SSM.
5. **MySQL** returns rows. Sequelize maps them to model instances. Lambda returns JSON with CORS headers.

→ *Follow-up: How did you handle connection pooling to MySQL from Lambda cold-starts?*

Lambda reuses execution environments across warm invocations. We initialize the Sequelize connection pool *outside* the handler function (at module load time) so it is reused. Pool max is set to 5 connections because Lambda scales by spawning new instances, not threads. We also set `pool.acquire` timeout to 3s and handle `SequelizeConnectionError` with a retry, since an idle Lambda can have its MySQL connection dropped server-side.

Q3 – Multi-Environment Config with SSM Parameter Store

Question: You deployed CampusMesh across 4 environments (local, dev, staging, production). What was different between environments, and how did you prevent staging config from leaking into production?

Answer: The Serverless Framework's `params` block maps parameter names to SSM paths that include the stage name – e.g., `${ssm:/dev/sqlldb/name}` for dev, `${ssm:/prod/sqlldb/name}` for prod. Deploying with `-stage prod` makes every SSM reference resolve to the `/prod/` namespace automatically. There is no mechanism for a dev deploy to pull prod credentials.

The four environments differed in: (1) **Database** – each had its own MySQL RDS instance, DynamoDB tables, and separate KMS keys. (2) **CORS** – local allowed `localhost:3000`; prod only allowed the CloudFront domain. (3) **Concurrency** – prod Lambdas had provisioned concurrency on high-traffic endpoints; dev/staging did not. (4) **Log retention** – prod kept 90 days; dev kept 7.

Additional safety: IAM deployment roles had different cross-account permissions per environment. A dev deploy script physically cannot touch the prod AWS account.

Q4 – Multi-Region CI/CD with CDK at Amazon

Question: At Amazon you built a multi-stage, multi-region CI/CD pipeline using AWS CDK with automated rollback controls. What triggered a rollback, and how did you validate it completed?

Answer: Two rollback triggers: (1) **CloudFormation stack failure** – if any resource update fails (Lambda code size exceeded, DynamoDB attribute mismatch), CloudFormation automatically rolls back the stack to the previous stable state. (2) **CloudWatch alarms** – 10-minute post-deployment bake window with alarms on error rate >1% and p99 latency >500ms. If an alarm fired during bake, the pipeline triggered rollback to the previous artifact version.

Validating completion: CloudFormation transitions to `UPDATE_ROLLBACK_COMPLETE`. The pipeline polled that status and sent an SNS alert with the stack ARN and rollback reason. We then hit a `/health` endpoint returning the deployed version hash – confirming the old version was live.

CDK vs raw CloudFormation: CDK is a TypeScript SDK that synthesizes CloudFormation templates. We wrote infrastructure as typed code with reusable constructs, parameterized the same stack across regions without copy-pasting YAML, and caught IAM policy mistakes at compile time (TypeScript type errors) rather than at deploy time.

Q5 – KMS Encryption and Least-Privilege IAM (Concrete Example)

Question: Give me a concrete example – what resource did a Lambda touch, and what exact IAM permissions did it have?

Answer: Concrete example from CampusMesh `study_group` service. The `membershipHandler` Lambda needs to: process queries via MySQL (DB connection, not IAM-gated), upload shared files to S3, and decrypt secrets via KMS. Its IAM policy had exactly three statements:

1. `kms:Decrypt` and `kms:GenerateDataKey` on this service's specific KMS key ARN – no wildcard.
2. `s3:PutObject` and `s3:GetObject` on `arn:aws:s3:::campusmesh-{stage}-files/*` – scoped to one bucket, not all S3.
3. `sqs:SendMessage` on the specific EventBridge SQS queue ARN – to emit group-joined events.

No `s3:DeleteObject`, no `sqs:DeleteMessage`, no `iam:*`. The recordings service had its own separate KMS key – a compromised `membershipHandler` cannot decrypt recording data. Over-granting (e.g., `Resource: "*" on S3`) means a compromised Lambda can exfiltrate data from every bucket in the account.

Q6 – Dependency Injection with Dagger at Amazon

Question: At Amazon you wrote Java backend services using Dagger for dependency injection. What is DI and why does it matter for testing?

Answer: Dependency Injection means a class *receives* its dependencies from outside rather than constructing them internally. Without DI, a service does `new DynamoDbClient()` in its constructor – in tests you hit real AWS. With DI, you pass the client in as a constructor parameter, so tests inject a Mockito mock instead.

Dagger does DI at **compile time** via annotation processing, not runtime reflection like Spring. Consequences: zero runtime DI overhead, and if you forget to provide a dependency the build fails immediately – not at application startup. At Amazon's scale, shifting failures left is critical.

In practice: annotate service classes with `@Inject`, define a `@Component` interface describing the object graph, Dagger generates the wiring code. For tests, write a separate test `@Component` that substitutes real clients with mocks – same dependency graph shape, different implementations.

Smithy defined the service model (operations, shapes, errors) in a `.smithy` file. The build plugin generated Java interfaces, request/response POJOs, and client stubs – my Java code called generated methods and Coral handled transport serialization automatically.

Q7 – 35% Execution Time Improvement at Amazon

Question: You cut backend execution time 35% at Amazon. What was the bottleneck and how did you find it?

Answer: Found via Amazon's internal profiling tool (equivalent to `async-profiler`). The flame graph showed two hot spots:

1. Serialization: The service used JSON for request/response payloads between internal components. Switched to **CBOR** (Concise Binary Object Representation) – a binary encoding of the same data model. Binary is 2–3× smaller on the wire and 3–5× faster to parse because there is no string tokenizing. This alone gave approximately 20% of the gain.

2. Object creation in tight loops: A helper class was instantiated on every loop iteration, triggering GC pressure. Applied **object pooling**: pre-allocate a pool of instances and reuse them. Eliminated mid-loop GC pauses that appeared as latency spikes under load.

The remaining gain came from **low-level design patterns**: lazy initialization (defer expensive computations to first use) and reordering conditional checks (fail-fast on the cheapest check first). After each change, I re-ran the profiler to confirm the hot spot shrank before moving to the next target. Never optimize without measurement.

Q8 – Unit Testing with Mockito (Java)

Question: How do you write a unit test for a Java method that calls DynamoDB? Show how you'd mock a ConditionalCheckFailedException.

Answer: Core principle: isolate the unit under test by replacing real clients with Mockito mocks. The method receives its `DynamoDbClient` via constructor injection, so the test injects a mock.

```
@Test
void updateRecord_whenConditionFails_throwsDomainConflict() {
    DynamoDbClient mockClient = Mockito.mock(DynamoDbClient.class);
    MyService service = new MyService(mockClient);

    when(mockClient.putItem(any(PutItemRequest.class)))
        .thenReturn(ConditionalCheckFailedException.builder()
            .message("Condition failed").build());

    assertThrows(ConflictException.class,
        () -> service.updateRecord("id-123", newData));
    verify(mockClient, times(1)).putItem(any(PutItemRequest.class));
}
```

`assertThrows` verifies the service translates the AWS exception into a domain-layer exception – not leaking AWS types to callers. `verify` ensures we didn't accidentally retry when we shouldn't have.

Mock vs stub: A stub returns canned data. A mock additionally tracks how it was called so you can **verify**. Use `verify` when the interaction itself (not just the return value) is part of the contract.

When to write an integration test instead: When the real behavior of the dependency matters – e.g., verifying a DynamoDB conditional expression evaluates correctly. A mock returns what you told it; the real DynamoDB evaluates the condition. Integration tests catch mismatches between your assumption and the service's actual behavior.

Q9 – WebSocket on Serverless Backend (Broker / NYU)

Question: Your Broker insurance platform uses WebSocket-based real-time communication. How did you maintain WebSocket connections on a stateless serverless backend?

Answer: AWS API Gateway's **WebSocket API** manages the persistent TCP connection on behalf of the backend – the Lambda does not hold it open. Each connected client gets a unique `connectionId` from API Gateway.

Connect: Client opens WebSocket. API Gateway fires the `$connect` route Lambda, which saves `{connectionId, userId, roomId}` to DynamoDB.

Message: Client sends a message. API Gateway fires the `$default` route Lambda. Lambda looks up all other `connectionIds` in the same room from DynamoDB and calls the **API Gateway Management API** to post to each: `POST /@connections/{connectionId}`.

Disconnect: API Gateway fires the `$disconnect` route Lambda, which deletes the record from DynamoDB.

Stale connections: If a mobile client drops without a close frame, the DynamoDB record remains but the `connectionId` is invalid. Calling the Management API returns **410 Gone**. The Lambda catches that status and deletes the stale record. The entire pattern is stateless from Lambda's perspective – state lives in DynamoDB.

Q10 – React Query vs Redux (NYU Insurance Platform)

Question: In the NYU insurance platform you used React Query alongside Redux Toolkit. When do you use React Query vs Redux for server state?

Answer: React Query owns server state; Redux owns client-only UI state.

Server state has a specific lifecycle: fetched asynchronously, potentially stale, needing background refresh, needed by multiple components. React Query handles all of this. `useQuery(['orders', customerId])` fetches, caches, deduplicates concurrent requests, and auto-refetches on window focus.

Redux is for state with no server counterpart (sidebar open/closed, active modal, wizard step) or state driven by complex client-side reducers. In the insurance platform, Redux held the active policy form state across a multi-step wizard; React Query held all API data (broker listings, carrier responses, document uploads).

Stale-while-revalidate: React Query returns cached data immediately so the UI never shows a blank loading state, while simultaneously fetching a fresh version in the background. When fresh data arrives, it updates the cache and re-renders. `staleTime` controls how long data is considered fresh before a background refetch triggers.

Optimistic updates and rollback: On mutation (submit a broker form), call `queryClient.setQueryData` with the expected result before the API returns. If the API fails, the `onError` callback calls `setQueryData` again with the snapshot saved in `onMutate`. Instant UI feedback with automatic rollback on failure.

Q11 – N+1 Queries in Sequelize (CampusMesh)

Question: CampusMesh uses Sequelize as an ORM over MySQL. What is the N+1 query problem and how do you prevent it?

Answer: N+1 occurs when you fetch N records and then run one additional query *per record* for associated data – totaling N+1 round trips. Example: fetch 20 study groups (1 query), then for each group fetch its members (20 queries) = 21 queries. With network latency, this is severe.

Prevention with eager loading:

```
const groups = await StudyGroup.findAll({
  where: { campusId },
  include: [{ model: Member,
    include: [{ model: User,
      attributes: ['id', 'name', 'avatar'] }] }]
});
```

Sequelize generates a LEFT JOIN (or a separate IN query), loading all data in 2 queries total.

Transactions: When a user joins a study group we insert a Membership row AND increment memberCount on StudyGroup. Both steps are wrapped in a Sequelize transaction – if step 2 fails, the insert rolls back. The EventBridge event only fires *after* the transaction commits; you do not want to emit an event for a membership that got rolled back.

When to drop to raw SQL: Complex aggregations with GROUP BY + HAVING, window functions, or bulk upserts where Sequelize generates inefficient queries. Use `sequelize.query()` with bound parameters (never string interpolation).

Q12 – REST API Design: Order History with Pagination

Question: Design a REST endpoint that fetches a customer's order history with pagination. URL, HTTP method, response body, and error format.

Answer: Endpoint:

```
GET /v1/customers/{customerId}/orders?limit=20&cursor=eyJpZCI6MTIzfQ
Authorization: Bearer <jwt>
```

Response 200:

```
{
  "data": [
    { "orderId": "ord_abc", "placedAt": "2026-06-09T14:00:00Z",
      "total": 149.99, "status": "shipped", "itemCount": 3 }
  ],
```

```
"pagination": { "nextCursor": "eyJpZCI6OTI9",
                "hasMore": true, "limit": 20 }
}
```

Error format:

```
{ "error": { "code": "FORBIDDEN",
            "message": "Cannot access another customer's orders",
            "requestId": "req-xyz" } }
```

Cursor vs offset: Offset pagination (`?page=3`) is broken for live data – if a new order inserts between page 1 and page 2 fetches, records shift and you miss or duplicate items. Cursor pagination encodes the last seen record's position; the next page picks up exactly where you left off regardless of inserts.

Status codes: 401 = unauthenticated (no valid token). 403 = authenticated but not authorized (valid token, wrong user). 404 = customerId doesn't exist. Never return 200 with `error: true` in the body.

Q13 – Idempotency for Order Placement

Question: What is idempotency and why does it matter for a POST endpoint like “place order”?

Answer: An operation is **idempotent** if running it multiple times produces the same result as running it once. GET and DELETE are naturally idempotent. POST (create order) is not – two requests create two orders.

Why it matters: a customer taps Buy on a slow connection. The request reaches the server, the order is created, but the response times out before reaching the client. The app retries. Without idempotency the customer gets charged twice.

Implementation:

1. Client generates a UUID per order attempt, sends it as a header: `Idempotency-Key: a1b2-c3d4`.
2. Server checks Redis or DynamoDB: has this key been processed?
3. If yes: return the *same* 201 response body from the first time – do not reprocess.
4. If no: process the order, store the response under the key with a 24-hour TTL, return 201.

The key is scoped to the customer: stored as `customer:{customerId}:idempotency:{key}` to prevent cross-customer collisions and limit blast radius of a leaked key.

Q14 – RAG Retrieval Mechanics (InviLite / CampusMesh)

Question: You built a RAG pipeline for the InviLite project and worked on LLM-based retrieval for CampusMesh CampusIQ. Explain the retrieval step – how does the system know which documents are relevant to an incoming query?

Answer: 1. Index time (offline): Documents (incident logs for InviLite; course materials and study group descriptions for CampusIQ) are split into chunks (512 tokens with 50-token overlap). Each chunk is passed through an **embedding model** (Bedrock for InviLite) which maps it to a dense vector in a high-dimensional space. Vectors are stored in a **FAISS** index.

2. Query time (online): The incoming query is embedded with the *same* model to get a query vector. FAISS computes **cosine similarity** between the query vector and all stored chunk vectors (dot product of two normalized vectors: equals 1 when vectors point the same direction, 0 when orthogonal). Top-K most similar chunks are retrieved.

3. Generation: Retrieved chunks plus the original query are inserted into the LLM context window via Bedrock. The LLM generates a response grounded in retrieved content rather than hallucinating from training data.

Dense vs sparse: Dense retrieval (FAISS/embeddings) captures semantic similarity – “EC2 instance unreachable” matches “host not responding” even with no keyword overlap. Sparse retrieval (BM25/TF-IDF) matches exact keywords. Hybrid retrieval combines both.

When RAG fails: (1) Retrieval misses the right chunk – embedding model fails on domain-specific jargon. (2) Retrieved chunks are noisy, contaminating the LLM context. (3) LLM ignores retrieved context – context window is too full. For InviLite, root-cause classification accuracy improved from 56% to 92% by switching from keyword search to embedding-based retrieval.

Q15 – Validating AI-Generated Code

Question: Nordstrom’s JD mentions “validating AI-generated code against requirements, security guidelines, and quality standards.” Walk me through your process – what do you trust automatically vs. always verify manually?

Answer: Trust without verification: Boilerplate scaffolding (Express route skeleton, Lambda handler shell), type definitions matching a known schema, refactoring to extract a helper, test file structure – things where correctness is visually obvious and covered by existing tests.

Always verify manually:

1. **Auth/authz logic** – AI sometimes omits ownership checks. Always ask: does this endpoint verify the requesting user owns this resource?
2. **SQL queries** – check for injection risk (parameterized vs concatenated), verify filters are correct (off-by-one in date ranges is a common AI mistake).
3. **IAM policies** – AI tends to suggest `Resource: "*"`. Always tighten to specific ARNs.
4. **Error handling** – AI often swallows exceptions silently. Every catch block should log, alert, or rethrow.
5. **Business logic with numbers** – rounding, currency arithmetic; I have caught AI doing floating-point division where integer arithmetic was required.

Real security issue caught: Copilot suggested a JWT verification snippet that used `algorithm: 'none'` as a fallback – which disables signature verification entirely and is a known attack vector. Caught it in code review before it merged.

Never let AI do unsupervised: Database migrations, access policy changes, anything touching payment flows.

Q16 – Properties of a Good Unit Test

Question: What makes a good unit test – what properties should it have?

Answer: A good unit test has the **FIRST** properties: **F**ast (milliseconds – no network, no disk, no sleep), **I**solated (tests one behavior; failures are unambiguous), **R**epeatable (same result every run regardless of environment or order), **S**elf-validating (passes or fails automatically), **T**imely (written alongside the code, not as an afterthought).

Test pyramid: 70% unit tests (fast, cheap, high logic coverage), 20% integration tests (verify components work together), 10% E2E tests (full user journey, slowest, most brittle). Over-indexing on E2E is a common mistake – they are expensive to maintain and slow CI.

Testing `Date.now()`: Inject a clock interface instead of calling `Date.now()` directly. In tests, pass a fake clock returning a fixed timestamp: `const clock = { now: () => 1717977600000 }`. This makes time-dependent logic deterministic.

Flaky tests: A test that sometimes passes and sometimes fails without code changes – usually caused by shared mutable state, timing assumptions (`sleep(100)`), or non-deterministic ordering. Fix by: fresh fixtures per test, replacing sleeps with proper awaiting, and sorting results before asserting order.

Q17 – Integration Testing Lambda + DynamoDB

Question: How would you write an integration test for a Lambda function that writes to DynamoDB without hitting the real AWS table?

Answer: Option 1 – DynamoDB Local / LocalStack: Run a local DynamoDB emulator in Docker as part of test setup. Configure the DynamoDB client to point to `http://localhost:8000` instead of AWS. The test:

1. **beforeAll:** create the table schema on the local emulator.
2. Invoke the Lambda handler function directly with a synthetic event object (no API Gateway needed).
3. **afterEach:** delete items created during the test, namespaced by test run ID.
4. **afterAll:** delete the test table.

Option 2 – Dependency injection with a test double: Inject the DynamoDB client via constructor DI and provide a lightweight in-memory implementation for the operations you use. Less fidelity but fast and no Docker dependency.

Limitation of local emulators: They do not perfectly replicate all DynamoDB behaviors – particularly conditional expressions, GSI consistency, and throttling. If your code relies on nuanced DynamoDB semantics, you need a real AWS integration environment.

Shared staging database risk: If multiple developers run integration tests against the same staging table simultaneously, data can collide. Fix: namespace test records by test run ID (e.g., PK: `test-{runId}-user-123`) and delete only records with that prefix in cleanup.

Q18 – PostgreSQL Locking in NYU ETL Pipeline

Question: In your NYU ETL pipeline you used concurrency control and locking when writing 250K+ records from NoSQL into PostgreSQL. What kind of locking and why?

Answer: We ran the ETL with parallel Python workers (`concurrent.futures.ThreadPoolExecutor`) to process data chunks concurrently. Without coordination, two workers could try to upsert the same record simultaneously, causing lost updates.

What we used: `INSERT ... ON CONFLICT DO UPDATE` (upsert) for the common case – atomic at the row level, no explicit lock needed. For records requiring read-then-modify (e.g., updating a running aggregate), we used `SELECT FOR UPDATE` within a transaction:

```
BEGIN;
SELECT * FROM health_metrics WHERE user_id = $1 FOR UPDATE;
UPDATE health_metrics SET score = $2 WHERE user_id = $1;
COMMIT;
```

Deadlock: Occurs when worker A holds lock on row X and waits for Y, while worker B holds Y and waits for X. PostgreSQL detects deadlocks automatically and kills one transaction with `ERROR: deadlock detected`. Prevention: always acquire row locks in the same order across workers (sort by `user_id` before locking).

Partial ETL failure: We batched records into chunks of 1,000, each wrapped in a transaction. A failed chunk rolled back cleanly. We logged the failed chunk's range to a `failed_batches` table and retried those specific ranges in a subsequent pass, avoiding reprocessing 180K successful records.

Q19 – DynamoDB vs MySQL: When and Why

Question: When would you use DynamoDB instead of MySQL, and what data model decisions change? How did you use both in CampusMesh?

Answer: **MySQL** for: relational data with complex queries (joins, aggregations), normalized schemas, ad-hoc querying, strict transactional consistency. **DynamoDB** for: high-throughput key-value or time-series access patterns known at design time, massive scale, no complex queries needed.

In CampusMesh:

- **MySQL (Sequelize):** Users, study groups, memberships, calendar events – relational data where you need “all groups in campus X with more than 5 members” (requires joins and filters).
- **DynamoDB:** WebSocket connection records, active call sessions, notification delivery status, real-time presence – high-throughput key lookups with short TTL, no complex queries needed.

Chat messages – single-table design: PK = ROOM#{roomId}, SK = MSG#{timestamp}#{messageId}. Query all messages in a room sorted by time with a single Query call: KeyConditionExpression: PK = :roomId AND SK BETWEEN :start AND :end.

Hot partition: If all writes hit the same partition key (viral room at 10K messages/sec), DynamoDB throttles that shard. Fix: write sharding across ROOM#{roomId}#SHARD#{0..9} and scatter-gather on reads. Or rely on DynamoDB’s Adaptive Capacity for automatic rebalancing.

Q20 – System Design: Product Search at Scale

Question: Design a simplified Nordstrom product search – a customer types “red dress size 8” and gets results in under 200ms. What components do you need?

Answer: Components:

1. **CDN (CloudFront):** Static assets served from edge. API responses are not CDN-cached – they are personalized and inventory-dependent.
2. **API Gateway + Search Service** (Node.js or Java): receives query, parses filters (color=red, size=8, category=dress), calls search index.
3. **OpenSearch/Elasticsearch cluster:** Pre-indexed product catalog. Query with a bool filter: exact match on size and color, relevance-ranked by full-text on description and title.
4. **Redis cache:** Cache the top 1,000 most common queries with a 5-minute TTL. Popular queries become sub-10ms.
5. **Inventory service:** Async sidecar – search results return first, availability badges update via a second lightweight call or WebSocket push.

Inventory without staling search: Product catalog (name, images, price) is updated in OpenSearch via SQS events on product change. Inventory (in-stock/out-of-stock) lives separately in DynamoDB, updated by warehouse systems in real-time, fetched in parallel with search and merged on the frontend. Search never blocks on inventory.

Black Friday 10x load: Auto-scaling OpenSearch cluster, pre-warmed EC2 capacity for the search service, Redis cluster mode for horizontal cache scaling, and **circuit breakers** – if OpenSearch is overloaded, fall back to a cached popular-items response rather than returning errors. CloudWatch alarm at 70% capacity triggers scaling before it breaks.

Part 2: Behavioral Questions with Detailed Answers

Format: Situation → Task → Action → Result. Values: CO = Customer Obsessed, OH = Owners at Heart, CEC = Curious & Ever Changing, HTW = Here to Win, WE = We Extend Ourselves.

B1 – Taking Ownership Beyond Your Scope [OH]

Question: Tell me about a time you took ownership of a problem that technically wasn't yours to fix.

Answer: At Amazon, my core project was the AFTPalletCVConsole – the React 18 frontend, Java Lambda backend, and RBAC integration. The CDK infrastructure stack (API Gateway, Lambda, DynamoDB, S3, IAM) was nominally shared responsibility, but the team member who owned it was pulled onto a higher-priority incident two weeks in, leaving the stack partially defined with no deployment pipeline.

Rather than wait and let my backend work sit undeployable, I picked up the CDK work: read Amazon's internal IaC guidelines, completed the Lambda function definitions, IAM least-privilege roles, DynamoDB table and GSI configuration, and the multi-stage CodePipeline setup. I presented the design to the team lead for review before deploying – I needed approval, not a bypass. They approved with minor tweaks to the IAM scoping.

Result: Infrastructure was live a week before the backend needed it. The CDK patterns I wrote were used as a reference for a subsequent internal tooling project. The lesson: if a dependency is blocking your work and you have the skill to unblock it, ask first – then do it. Don't wait.

B2 – Changing Based on User Feedback [CO]

Question: Describe a time when you built something and then changed it based on user feedback. What did you learn?

Answer: At CampusMesh, I built the initial CampusIQ AI assistant using keyword-based search over study group names and descriptions. Student feedback was consistent: "It only finds groups with the exact word I typed. If I search for 'machine learning' it misses groups that say 'ML' or 'AI fundamentals'." The search felt brittle and literal.

I shifted the architecture to embedding-based semantic search: chunked study group metadata through an embedding model, indexed into FAISS, and rewrote the query path to do vector similarity search. "Machine learning" then matched "neural networks," "deep learning," and "AI study group" by semantic proximity.

Result: Contextual accuracy improved 30%. User retention on CampusIQ sessions went up measurably. The lesson: keyword search feels obvious to an engineer but feels broken to a user. You only learn what "working" actually means by watching users, not by reading your own code.

B3 – Learning a New Technology Quickly [CEC]

Question: Tell me about a technology you had to learn quickly for a project. How did you approach it?

Answer: At Amazon I had never used Coral, Smithy, or Dagger before the internship – Amazon-internal frameworks with limited external documentation. I needed to be productive within two weeks.

My approach: (1) Read existing service implementations in the codebase first – real working code is more reliable than wikis. (2) Build a tiny throwaway service using all three before touching production code – a "hello world" that defined a Smithy model, wired a Dagger component, and made a Coral call to DynamoDB. (3) Ask one senior engineer for a 30-minute walkthrough of the Dagger component graph – not to be taught from scratch, but to validate my mental model before I built on it.

Result: Contributing independently to the real service within 10 days. The principle: don't read docs front-to-back. Find a real example, replicate it at small scale, identify what you don't understand, and fill those specific gaps.

B4 – Getting the Team Back on Track [HTW]

Question: Tell me about a time your team was behind and you helped get the project back on track.

Answer: At PwC, mid-sprint on a client onboarding feature, two engineers went on planned leave simultaneously. We had committed to three user stories. I took on two stories myself (the ones I understood best from the design doc) and reorganized the remaining work so the other engineers were unblocked and not context-switching.

I also introduced a daily 15-minute standup for that sprint – not to micromanage, but to surface blockers immediately. One standup caught an Azure API dependency issue that would have cost two days; we caught it on day one. We delivered two of the three stories on time. The third was descoped to the next sprint, communicated proactively to the client.

Result: Client satisfaction held. No defects in what we shipped. The lesson: when a team is behind, the first move is to remove ambiguity and blockers, not just to work harder.

B5 – Helping a Teammate Without Being Asked [WE]

Question: Describe a time you helped a teammate who was struggling, without being asked.

Answer: At PwC, a junior engineer (six months in) was assigned to deploy an Azure Function App as part of our CI/CD pipeline. She spent two days without success. I noticed her Jira ticket hadn't moved and asked if she wanted a second pair of eyes.

We pair-programmed for two hours. The issue was the event binding in `function.json` was pointing to the wrong storage queue name – a subtle environment-specific config mismatch. We fixed it together, and I walked her through how I diagnosed it: check Azure Monitor logs first, then trace the event routing. Afterward I wrote a HOW document for the team wiki documenting the correct pattern.

Result: That document became the reference for three more engineers during onboarding. I didn't help because I was asked – I noticed the signal (stalled ticket, unusual quiet) and acted on it. Helping isn't always a direct request; sometimes it's reading a situation and choosing to show up.

B6 – Decision with Incomplete Information [OH]

Question: Walk me through a decision you made with incomplete information. What was your reasoning and what was the outcome?

Answer: At Amazon, I was designing the data console for FC teams. I needed query volume estimates to size the backend – but operational data from the FCs was not available to me. Waiting would have delayed the design review by three weeks.

I made an explicit, documented assumption: “Assuming 50 FCs, 10 authorized users each, 20 queries/day average = 10,000 queries/day, with a 5x spike during active investigations = 50,000 queries/day design ceiling.” I surfaced this prominently in the design doc. The design used Lambda with reserved concurrency and DynamoDB on-demand mode so it would handle 10x that ceiling if the assumption was wrong.

Result: Design approved. Actual volume when data came in: 8,000 queries/day – within 20% of my estimate. The outcome was fine not because my estimate was perfect, but because I designed for uncertainty. Lesson: when you lack information, make your assumptions explicit and design for robustness rather than precision.

B7 – Technical Decision with Direct Customer Impact [CO]

Question: Tell me about a time a technical decision you made had a direct impact on a customer or end user.

Answer: At NYU’s Novel AI Technologies, insurance platform brokers submitted policy applications by email – PDFs sent to carriers, replies by PDF, data manually re-keyed. I built real-time carrier platform integration: brokers fill a structured form, it syncs to carrier systems via API, responses appear in the portal within minutes.

Three specific decisions that drove the impact: (1) WebSocket-based real-time communication so brokers do not refresh to see carrier responses. (2) Document processing pipeline so brokers upload supporting docs directly, with data auto-extracted into the form. (3) Row-level security in PostgreSQL so each broker sees only their own clients’ data.

Result: Six insurance clients onboarded. \$50K+ revenue generated. A broker told us their submission-to-quote time dropped from 3 days to 4 hours. That is what makes technical decisions feel real.

B8 – Disagreeing with the Technical Approach [CEC]

Question: Describe a situation where you disagreed with an approach and advocated for a different technical direction.

Answer: At PwC, the team planned real-time case status updates using polling – the frontend calling an API every 5 seconds. I disagreed: 5,000 customers at 1 call/5s = 1,000 req/min at baseline, unnecessary DB reads, and a 5-second lag still visible to users.

I proposed event-driven webhooks: when a case status changes in Dynamics CRM, a Power Automate flow fires a webhook to our backend, which updates the DB and pushes a WebSocket notification to the connected client. I built a PoC in two days: a webhook receiver, a DynamoDB update, and a test WebSocket notification to a frontend. I presented the PoC with a latency comparison (5s vs sub-second) and a resource cost estimate.

Result: Team adopted it. Case update latency dropped from 5 seconds to under 1 second. Manual case entry reduction hit 70%. Lesson: disagreement without a PoC is just opinion. A PoC turns a preference into evidence.

B9 – Most Complex Technical Project [HTW]

Question: Tell me about your most complex technical project. What made it complex and what would you do differently?

Answer: The Amazon internship: four parallel workstreams in 12 weeks – the Smithy service model and Java Lambda backend (Coral, Smithy, Dagger, CBOR), the React 18 frontend (React Harmony), the RBAC integration (Midway SSO, HELIS, OAuth), and the CDK multi-stage CI/CD infrastructure. All four had hard dependencies: the frontend could not be built without finalized API shapes from Smithy; the backend could not be tested end-to-end without CDK deployed; the RBAC integration blocked on the identity service team’s availability for joint testing.

What made it genuinely complex: I was learning Amazon-internal frameworks (Coral, Smithy, Dagger, HELIS) simultaneously, and every architectural decision required design review approval. A slip in one component compressed the schedule on everything downstream.

What I would do differently: Define and freeze inter-component contracts on day one – Smithy model shapes, API request/response structures, DynamoDB key schema – before any implementation begins. In week 5, a DynamoDB key naming change propagated across the Java backend, the React Query cache keys, and the CDK table definition simultaneously. Freezing the schema earlier would have avoided that churn.

B10 – Owning a Mistake [WE]

Question: Tell me about a time you made a mistake at work. What did you do?

Answer: At PwC, I deployed a Power Automate flow intended to run batch updates on imported records. I misconfigured the trigger – it fired on *every individual record update* instead of only on batch import completion. During the next large import (100K records), the flow triggered 100,000 times simultaneously, causing Azure API throttling errors and taking down a downstream notification system for 45 minutes.

I identified the root cause within 20 minutes via Azure Monitor logs, disabled the flow, cleared the backlog, confirmed the downstream system recovered. Then I wrote a root-cause analysis document: what happened, why, what I should have caught (no high-volume testing of the trigger), and what controls would prevent recurrence.

Result: Controls implemented: trigger conditions explicitly checked for batch completion; high-volume load testing added as a required pre-deployment step; this failure mode added to the code review checklist. Production defects from similar mistakes dropped 25% over the following two quarters. Owning the mistake publicly built more trust than the incident cost.

B11 – Prioritizing Under Deadline [OH]

Question: Describe a time you had to prioritize competing tasks under a deadline.

Answer: At Amazon, the final two weeks before intern demo day, three deliverables: the Java Lambda backend with HELIS integration (primary – core functionality), the React 18 frontend (secondary – the visible demo surface), and CDK pipeline finalization with multi-region promotion (foundation – nothing deploys without it). All three due before demo.

I applied dependency-first priority: (1) CDK infrastructure first – without a deployed stack, neither the backend nor frontend could be tested end-to-end. A day lost here costs a day on everything downstream. (2) Java backend second – core RBAC logic, Smithy operations, pre-signed URL flow – highest demo correctness risk. (3) React frontend last – could demo with stubbed API responses if the backend was incomplete, and the UI does not block backend functionality.

I communicated the priority order to my manager explicitly: “If we hit the wire and I have to cut something, I will cut frontend polish, not backend correctness or security.” He agreed. **Result:** All three delivered before demo day. The key was making the cut order explicit and agreed-upon early – no last-minute negotiation under pressure.

B12 – Proactively Flagging a Risk [CO]

Question: Tell me about a time you proactively flagged a risk or issue before it became a problem.

Answer: Before CampusMesh launched study group features publicly, I was load-testing Lambda functions. The `recordingHandler` Lambda had cold-start latency of 3–5 seconds – infrequently invoked, so almost always cold. A user clicking “Start Recording” would see a 4-second freeze with no feedback.

I flagged it to the team before launch: “This function is correct but will feel broken to users because of cold-start behavior.” I proposed and implemented **provisioned concurrency** for the recording Lambda – keeps one warm instance ready at all times. Cost: approximately \$8/month. Benefit: sub-200ms response on every invocation.

Result: We added this as a checklist item: every Lambda in a user-facing, latency-sensitive path gets evaluated for provisioned concurrency before launch. The issue never reached a user because we found it in testing. That is the best outcome – invisible to users precisely because we looked.

B13 – Adopting a New Tool for a Better Solution [CEC]

Question: Tell me about a new tool or framework you adopted to solve a problem differently.

Answer: For the InviLite project at NYU, the initial design used Elasticsearch keyword search for incident log retrieval. I knew keyword search would struggle because engineers describe the same failure in many ways (“host unreachable,” “connection timeout,” “node not responding” – same underlying issue).

I proposed switching to embedding-based semantic search using FAISS and Bedrock’s embedding models – tools I had read about but not used in a production pipeline. Two weeks of learning: FAISS docs, experiments on small log datasets, understanding index types (Flat vs IVF vs HNSW) and their trade-offs for our dataset size.

Result: Root-cause classification accuracy jumped from 56% to 92%. Incident resolution time dropped from 1–2 hours to minutes. Choosing the right retrieval technology – even at the cost of a learning curve – was the highest-leverage decision in the entire project.

B14 – Receiving Critical Feedback [HTW]

Question: Tell me about a time you received critical feedback and how you responded.

Answer: During a code review at Amazon, my tech lead flagged that my CDK stack had overly broad IAM policies: I had used **Resource: "*" on a DynamoDB access statement** – the Lambda could read any DynamoDB table in the account. He was direct: “This is a security issue, not a style issue. It needs to be fixed before this merges.”

I thanked him, did not get defensive, and asked him to walk me through the correct ARN pattern. Then I audited *every* IAM statement in the CDK stack – not just the one he flagged. Found two more with overly broad resources. Fixed all three, added a code comment explaining the correct scoping pattern.

Result: “Specific ARN, not wildcard” became a personal checklist item for every IaC change I write. Critical feedback is the most useful feedback you can get – it tells you exactly what to improve. The right response is to fix it, learn it, and turn it into a habit.

B15 – Explaining Technical Concepts to Non-Technical Stakeholders [WE]

Question: Describe a time you had to explain a complex technical concept to a non-technical stakeholder.

Answer: At PwC, we were deploying an NLP-based OCR model to automate document data extraction for a client. The model required 3–4 weeks of training on the client’s specific document formats. The business operations director pushed back: “Why does software need training? Can’t you just turn it on?”

I explained: “Imagine hiring a new employee to read and process your specific forms. On day one, they can read English – that is the base model. But they do not know your forms or what fields matter. Training is the week they spend learning your documents before you trust them to process a thousand per day. Once trained, they handle 1,000 forms in the time it used to take to learn from 10. The 4-week training is exactly that onboarding period.”

Result: The client immediately understood and approved the timeline. They used the same analogy in their own internal presentations. The principle: find a concept the stakeholder already understands and map the technical concept onto it. Do not simplify – translate.

B16 – Making a Process Better [OH]

Question: Tell me about a time you made a process better – not just the code, but how the team works.

Answer: At PwC, code reviews were happening but inconsistently. No shared standard, so defects slipped through in different areas depending on who reviewed. Production bugs were occurring at 4–5 per sprint.

I introduced a structured code review checklist – a one-page reference with four categories: (1) Security (injection risks, auth checks, secrets in code), (2) Performance (N+1 queries, unbounded loops, missing indexes), (3) Test coverage (unit test exists, edge cases covered), (4) Naming and readability. I presented it as a tool, not a mandate: “This is what I check – let’s see if it helps.”

Result: Within two sprints, production defects dropped from 4–5 to 1–2 per sprint. Over the following quarter, aggregate defect rate fell 25%. The checklist became the onboarding standard for code quality. Process improvement multiplies across everyone on the team.

B17 – Speed vs Quality [CO]

Question: How do you balance shipping fast with shipping quality? Give me a real example.

Answer: At CampusMesh, there was pressure to launch real-time study group chat quickly. Building a production-quality WebSocket system (connectionId management, fan-out, stale connection cleanup) would take 3–4 weeks. Polling-based chat: 1 week.

My approach: ship polling first, but architect the handler to be swap-able. I defined a clean interface for the messaging layer (`sendMessage(roomId, message)`, `getMessages(roomId, cursor)`), implemented it with polling underneath, and documented exactly where the WebSocket swap would happen. Users got chat in week 1. Two weeks later, I swapped the implementation to WebSocket without touching a single line of frontend code.

Result: Users got the feature fast. The WebSocket upgrade was smooth. Key distinction: shipping polling chat was fast *and* disciplined – it worked correctly, had unit tests, had error handling. What I did not do was hardcode polling assumptions everywhere so the WebSocket upgrade would require a rewrite. Balance speed and quality by making intentional, reversible trade-offs.

B18 – Experiment That Didn’t Work [CEC]

Question: Tell me about a time you experimented with something that didn’t work. What did you do next?

Answer: At NYU’s AI health platform, I tried to fine-tune a DistilBERT model for health metric classification – predicting whether a wearable reading indicated a health risk. The hypothesis: a fine-tuned small model would be cheaper to run and fast enough for real-time Lambda inference.

After fine-tuning on our labeled dataset of 3,000 examples, accuracy was 71% – below the 80% threshold needed for clinical usefulness. I ran failure analysis: the model was missing sequential context – a spike is fine if preceded by exercise; alarming if at rest. DistilBERT with single-reading input could not capture that.

Rather than fine-tuning longer (more data would help but we did not have it), I pivoted to few-shot prompting with a larger model via Bedrock API, passing the last 5 readings as context. Accuracy hit 84%, exceeding the threshold.

Result: Higher per-inference cost but acceptable at our volume. Lesson: set an explicit exit condition before starting an experiment. When the result fell short, I diagnosed *why* – found the root cause – and changed the approach rather than just tuning harder.

B19 – Collaborating with a Different Working Style [HTW]

Question: Describe a time you had to collaborate with someone with a very different working style.

Answer: At PwC, I worked closely with a senior engineer who was a specification-first architect: detailed functional spec, then technical design doc, then code. My instinct is to prototype quickly and let the design emerge from working code.

We were assigned a new data integration feature together. Rather than each operating in frustration, I proposed a negotiated approach: I would build a throwaway prototype in two days (no production code – just enough to expose the real design questions), then we would write the technical design doc together using the prototype’s findings to make it concrete. He would then apply his detail-oriented review before we wrote any production code.

Result: Better than either approach alone. The prototype surfaced three design questions we would have gotten wrong in a spec-only process. His thorough review caught a data consistency issue I would have missed prototyping alone. Working style differences are a feature when you negotiate how to combine them.

B20 – Why Nordstrom, Why This Team [OH + CO]

Question: Why Nordstrom specifically, and why the Customer Experience Platform team?

Answer: Most of my technical work has been on platforms serving internal users (Amazon FC teams) or niche professional users (insurance brokers, health researchers). Nordstrom’s Customer Experience Platform is different: every API I would write is a touchpoint between Nordstrom and a real customer making a purchase decision – something they care about personally.

That shift from internal tooling to consumer-facing systems is exactly where I want to develop. The stakes are direct: a search API 200ms slower is not just a performance metric – it is a shopper who loses patience and leaves. A broken recommendation is a customer who did not find what they were looking for. I want to build systems where the feedback loop between engineering and customer experience is that immediate.

The platform sits at the intersection of backend engineering (microservices, cloud), data (real-time inventory, personalization), and AI (search relevance, recommendations) – exactly where my skills from Amazon (Java backend, AWS infrastructure, system design for internal tooling) and CampusMesh (Node.js, real-time serverless) overlap most directly.

Nordstrom’s values also resonated: *Customer Obsessed* and *Owners at Heart* describe how I have tried to work. Moving with speed while building customer trust is not a trade-off – it is how good engineering works when you build it right from the start.

Part 3: Deep Technical Questions – CampusMesh (10)

Questions grounded directly in the codebase: Sequelize models, DynamoDB message model, WebSocket handler, call lifecycle, IAM policy files, and the serverless.yml architecture.

CM1 – Dual Database Architecture: Why MySQL AND DynamoDB?

Question: CampusMesh uses MySQL (via Sequelize) for study spaces and members, but DynamoDB for messages. Why did you split across two databases instead of keeping everything in one?

Answer: This was a deliberate data-access-pattern decision, not a technology preference. Study spaces, members, invitations, and events are **relational data**: you frequently need to join them (“give me all groups in campus X where the requesting user is APPROVED”), run filters across multiple columns, and maintain referential integrity (a member record must reference a valid space). MySQL with Sequelize handles all of that naturally.

Messages are fundamentally different: the only access pattern is “**all messages for space X, sorted by time, paginated.**” You never need to join messages to members or filter by arbitrary fields. DynamoDB’s GSI-based Query is purpose-built for that pattern – you get sub-10ms retrieval at any message volume, with automatic horizontal scaling, without writing a single index or thinking about table partitions.

The trade-off you accept with DynamoDB: you must decide your access patterns at design time. We also store reactions, read receipts, and delivery status inside the message item itself (as arrays) rather than in separate tables – because DynamoDB can’t join. That works fine because reactions are always fetched with the message, never independently.

Had we stored messages in MySQL, we’d have had to manage connection pool exhaustion under high message volume, add custom pagination logic with LIMIT/OFFSET (which gets slow as offsets grow large), and manage message table indexing carefully. DynamoDB makes none of that our problem.

→ *Follow-up: The DynamoMessage model creates a composite key field space_id_created_at as a string: ‘\$space_id#\$timestamp’. Why store this as a denormalized string field?*

This is the GSI sort key pattern. DynamoDB GSIs require a partition key and optionally a sort key – both must be scalar attributes (string, number, or binary). To sort messages within a space by time, we need `space_id` as the GSI partition key and `created_at` as the sort key. But we also need to filter deleted messages and support thread queries. The compound field `space_id_created_at` pre-computes the GSI composite key at write time so that the `space-id-created-at-index` GSI query is: `KeyConditionExpression: space_id = :id` with `ScanIndexForward: false` (newest first). Writing the compound key once at insert time avoids a compute step on every read.

→ *Follow-up: The repository uses offset-based pagination by scanning all messages and slicing in application code. What’s the problem with this and how would you fix it?*

The code literally fetches *all* messages for a space in a do-while loop, then does `allMessages.slice(offset, offset+limit)` in JavaScript. This is an O(N) DynamoDB read that costs money and time proportional to total messages in the space. A space with 50,000 messages would read 50,000 items to return 20. Fix: use DynamoDB’s native **cursor pagination** via `ExclusiveStartKey`. The first query returns `LastEvaluatedKey`; the next page passes that as `ExclusiveStartKey`. No items are read that don’t appear in the result. Encode `LastEvaluatedKey` as a base64 JSON cursor, pass it opaquely to the client, and decode it on the next request.

CM2 – WebSocket Handler: In-Memory Map in Serverless

Question: Your websocketHandler.js stores active WebSocket connections in `new Map()` inside the class. What fundamental problem does this create in a serverless Lambda environment?

Answer: This is a critical architectural flaw that exists in the current implementation: an in-memory `Map` is **per Lambda instance**, not global. Each Lambda invocation runs inside an execution environment (a container). When

ten users connect simultaneously, AWS scales to ten Lambda instances. Each instance has its own `Map` with only the connections it personally accepted – no instance has visibility into other instances’ connections.

Consequence: when Lambda instance A tries to broadcast a message to all users in a study space, it iterates over *its own* `Map`, finds maybe 1–2 connections, and misses the other 8 users sitting in other Lambda instances. From a user’s perspective: some people receive messages, others don’t – and it’s non-deterministic, which is the worst kind of bug.

Correct fix: externalize connection state to DynamoDB. On `$connect`: Lambda writes `{connectionId, userId, spaceId, ttl}` to a DynamoDB connections table. On message broadcast: Lambda queries DynamoDB for all `connectionIds` with the target `spaceId`, then calls the API Gateway Management API (`@connections/{connectionId}`) for each. On `$disconnect`: Lambda deletes the DynamoDB record. Every Lambda instance reads from the same DynamoDB table, so the broadcast is correct regardless of how many instances are running.

→ *Follow-up: The `websocketHandler` also handles `TYPING_INDICATOR` messages with `broadcastToSpace`. What happens to latency if you have 500 users in a study space all typing simultaneously?*

Broadcasting a typing indicator to 499 other connections requires 499 sequential `@connections` API Gateway calls. At 1–5ms per call, that’s 0.5–2.5 seconds of serial I/O per typing event. Fix: parallelize with `Promise.all()` so all 499 posts fire concurrently. Also, typing indicators should be debounced client-side (send at most one event per 300ms) and dropped after 3 seconds with no update – they are ephemeral, losing them is acceptable, but drowning the backend with them is not.

CM3 – Message Send + WebSocket Broadcast: Decoupled Failure

Question: In `sendMessageService.js`, after writing the message to DynamoDB, the WebSocket broadcast is wrapped in a try-catch that logs a warning but does NOT throw. Why is this the right design? When would it be wrong?

Answer: This is the **fail-gracefully pattern** applied correctly to a two-tier operation. The message write to DynamoDB is the *source of truth* – if it succeeds, the message exists and will be delivered. The WebSocket broadcast is a *real-time convenience* – it pushes the new message to currently connected clients so they don’t have to poll. If the broadcast fails:

- The message is still in DynamoDB.
- Any client that polls (e.g., on page load or reconnect) will receive the message.
- Only the real-time push is missed, which means a small delay in delivery, not data loss.

Swallowing the WebSocket error is correct because: (1) WebSocket delivery is best-effort by nature, (2) a transient API Gateway failure shouldn’t roll back a successfully persisted message, and (3) the client has fallback polling to recover.

When it would be wrong: if the WebSocket broadcast were the *only* delivery mechanism and there were no fallback. In that case, a silent failure means the user never sees the message at all. The design is only safe because DynamoDB is the durable store and the broadcast is a supplementary path.

CM4 – User ID Encryption: Why Deterministic?

Question: The codebase uses `UserIdEncryption.encryptUserIdDeterministic()` before storing user IDs in MySQL and DynamoDB. Why encrypt user IDs at rest, and what does “deterministic” mean here – isn’t deterministic encryption weaker?

Answer: User IDs in this context come from an external identity provider (the auth JWT). Storing them in plaintext means anyone with read access to the database (a misconfigured query, a future engineer, a leaked backup) can directly correlate platform activity with real identities. Encrypting user IDs at rest adds a layer of defense-in-depth: database records are pseudonymous even if exfiltrated.

Why deterministic: We need to do equality lookups – “find all members where `user_id = X`” (see the `WHERE user_id = encryptedUserId` in `MemberModel`). Standard encryption is randomized: the same plaintext produces a different ciphertext each time (due to a random IV), so two encrypted versions of the same user ID would look different. That breaks equality-based SQL `WHERE` clauses.

Deterministic encryption (also called “format-preserving” or “SIV” encryption) always produces the same ciphertext for the same plaintext input. This allows SQL equality lookups while keeping the field opaque to readers who don’t have the key. The trade-off: it leaks whether two rows belong to the same user (because they produce identical ciphertext). For our use case – equality lookups on user membership – that trade-off is acceptable.

The real weakness: if the encryption key in the KMS/SSM store is compromised, all user IDs can be decrypted in bulk. The key rotation strategy and KMS audit logs are therefore critical.

CM5 – Sequelize Pool Configuration in Lambda

Question: Your `dbConfig.js` sets `pool.max = 5`, `pool.idle = 10000`. Why those numbers? What happens if you set `pool.max` to 50?

Answer: Lambda scales horizontally by spawning new *instances* (execution environments), not by creating more threads within one instance. Each Lambda instance is single-threaded (Node.js is single-threaded event loop). So each instance only needs **one active DB connection** at a time – it processes one request, returns, then handles the next. Pool max of 5 gives a small buffer for rare cases where async code overlaps multiple queries in the same invocation.

If you set `pool.max = 50`: at 100 concurrent Lambda invocations, that’s up to 5,000 potential MySQL connections. MySQL’s default `max_connections` is 151. The 52nd Lambda invocation would start failing with **Too many connections**. This is the classic **Lambda-RDS thundering herd problem** – one of the main reasons managed Lambda connection pooling solutions like RDS Proxy exist. RDS Proxy sits between Lambda and MySQL, maintains a stable pool of connections to the DB, and multiplexes Lambda connections through it.

`pool.idle = 10000`: if a connection goes unused for 10 seconds, the pool releases it. This prevents holding MySQL connections open during low-traffic periods when the Lambda may be warm but idle.

→ *Follow-up: What is Lambda cold-start latency in this context and how do you mitigate it?*

Cold-start: when a new Lambda instance is created (no warm instance available), it must download the code, start the Node.js runtime, and execute module-level initialization (which includes creating the Sequelize connection pool and connecting to MySQL). This adds 500ms–3s to the first request. Mitigation: (1) provisioned concurrency – keeps N instances pre-warmed (used for `recordingHandler` in `CampusMesh`), (2) keep module-level initialization minimal and do lazy initialization for rarely-used paths, (3) use connection pooling (already done) so the pool creation cost is amortized across warm invocations.

CM6 – Call Lifecycle: AWS Chime + Auto-End Timer

Question: The `initiateCallService` creates an AWS Chime meeting and sets an `autoEndTime`. How does the auto-end timer actually work in a serverless architecture? There’s no long-running process to fire it.

Answer: AWS Chime SDK creates a meeting with a `meeting_id` and a `MediaRegion`. The `autoEndTimerMinutes` (defaulting to 30) is stored in the call record in MySQL (in the `StudySpaceCall` table with `auto_end_time` calculated from `started_at`). The challenge is: Lambda instances die after their request completes – there’s no persistent process to fire a timer 30 minutes later.

In `CampusMesh`, there’s an `autoEndCallsHandler.js` in the `handlers` directory. This is a **scheduled Lambda** – triggered by EventBridge on a cron schedule (e.g., every 5 minutes). It queries MySQL for all calls where `status IN ('INITIATED', 'CONNECTED')` AND `auto_end_time < NOW()`, then calls the Chime `DeleteMeeting` API for each, and updates the call status to `ENDED`. This is the **polling scheduler pattern**: instead of setting individual timers, a periodic sweep cleans up expired state.

Latency implication: calls aren't ended at exactly the 30-minute mark – they're ended within the next cron period (up to 5 minutes late). That's acceptable for a study session auto-end. If you needed exact timing, you'd use EventBridge Scheduler to create a per-call scheduled event that fires at `auto_end_time` exactly.

CM7 – StudySpace Data Model: Encryption Hooks

Question: The StudySpace Sequelize model applies encryption/decryption hooks via `sequelizeHooks.beforeCreate(Schema, ['name', 'description'])`. Why encrypt those specific columns, and what's the performance implication of running crypto on every write and read?

Answer: Study space names and descriptions can contain sensitive information about academic topics, student groups, and health conditions (e.g., a space named “HIV research support group”). Encrypting at the application layer – as opposed to transparent database encryption – means that even a direct DB read by a DBA or a leaked SQL export returns ciphertext, not plaintext. This is row-level field encryption, not full-disk encryption.

The **Sequelize hook approach** ensures encryption/decryption is transparent to service layer code: service code reads and writes `space.name` as plaintext. The hook fires before the SQL is generated (BeforeCreate/BeforeUpdate) to encrypt, and after the result is returned (AfterFind) to decrypt. This pattern keeps crypto concerns out of business logic.

Performance implication: AES encryption on a single string field takes microseconds – it is negligible compared to network I/O to MySQL. The real cost is not crypto; it's that encrypted columns **cannot be indexed for range queries or partial matches**. You cannot do `WHERE name LIKE '%machine learning%'` on an encrypted column (the ciphertext doesn't match). This is why we use embedding-based search (FAISS) for the CampusIQ search feature instead of SQL LIKE queries – you can't efficiently search encrypted text.

CM8 – Role-Based Access in the Call System

Question: The recording handler checks that the requester is either the call initiator OR an ADMIN member of the study space. Walk me through how that authorization check works across two DB tables, and what race condition could exist.

Answer: The check makes two sequential DB queries: (1) `StudySpaceMember.findOne({where: {space_id, user_id: encryptedUserId, status: 'APPROVED', is_deleted: false}})` to confirm the user is a valid member and retrieve their role. (2) `StudySpaceCall.findOne({where: {call_id, space_id, status: ['INITIATED', 'CONNECTED']}})` to get the call and its `initiator_id`. Then the code checks: `call.initiator_id !== encryptedUserId && membership.role !== 'ADMIN'` – if both are true, 403.

Race condition: between query 1 (membership check) and the recording action, the user's role could be changed by another admin (e.g., demoted from ADMIN to MEMBER). The membership we validated is now stale. In a single-threaded, in-process system this wouldn't happen, but with concurrent Lambda invocations it can. Mitigation: wrap both queries and the action in a MySQL transaction with `SELECT FOR UPDATE` on the membership row, holding the row lock until the recording action completes. This prevents concurrent modification of the membership row between the read and the action.

CM9 – S3 Pre-Signed URLs for Shared Files

Question: The `sharedFilesHandler` handles file uploads for study groups. What is a pre-signed URL pattern and why is it the right approach for file uploads in a serverless architecture?

Answer: In a naive implementation, files would flow through the Lambda: client uploads to Lambda endpoint, Lambda receives the bytes, Lambda uploads to S3. This is wrong for several reasons: (1) Lambda has a 6MB request body limit (via API Gateway), far too small for video files; (2) routing all file bytes through Lambda is expensive (compute time + data transfer costs); (3) Lambda has a 15-minute max execution time, which can be hit by large uploads on slow connections.

Pre-signed URL pattern: (1) Client calls `GET /study-groups/{id}/upload-url?fileType=image/jpeg`. Lambda generates a **pre-signed S3 PutObject URL** using the AWS SDK – a time-limited, signed URL that allows one specific PUT to one specific S3 key. Lambda returns this URL to the client. (2) Client uploads the file directly to S3 using the pre-signed URL – Lambda is not in the data path at all. (3) After upload, client calls Lambda to record the file metadata (S3 key, size, type) in MySQL.

S3 handles the actual bytes: no Lambda size limits, no compute cost for data transfer, and S3 can handle multi-part uploads for very large files. The pre-signed URL expires (typically 15 minutes) so it cannot be reused or shared outside the upload window. The Lambda-issued URL embeds the IAM signature – the client never sees AWS credentials.

CM10 – Soft Delete vs Hard Delete: Membership and Messages

Question: Multiple models – `StudySpaceMember`, `StudySpace`, and messages – use an `is_deleted` soft-delete flag instead of removing the row. Explain the design trade-off and what you need to do on every query to make this work correctly.

Answer: Why soft delete: (1) *Audit trail* – you can answer “did user X ever belong to study space Y?” even after they left, which is important for moderation and reporting. (2) *Data recovery* – accidental deletion is reversible; flip `is_deleted` back to false. (3) *Referential integrity* – if a message references a member who is hard-deleted, foreign key constraints would either block the delete or cascade-nullify the reference. Soft delete avoids that complexity. (4) *Analytics* – historical data stays queryable for usage reports.

What you must do on every query: every `findAll`, `findOne`, and `count` on soft-deleted tables must include `WHERE is_deleted = FALSE` (or `{where: {is_deleted: false}}` in Sequelize). If you forget this on any query, you silently return deleted records to users. This is a footgun. Mitigation options: (1) Define a Sequelize **default scope** on the model: `defaultScope: {where: {is_deleted: false}}` – it applies automatically to every query unless explicitly unscoped. (2) Use a Sequelize **paranoid** option which adds soft delete natively and manages the filter for you. The current codebase adds the filter manually per query, which requires discipline across every developer.

The DynamoDB side: messages use `is_deleted: false` filtered in *application code* (after DynamoDB returns results). This is expensive – deleted messages are still read from DynamoDB and paid for. Better: use a DynamoDB **FilterExpression** to avoid returning them over the network, or use a TTL attribute to let DynamoDB purge them automatically after a retention window.

Part 4: Deep Technical Questions – Amazon SDE Internship (10)

Questions grounded in the AFTPalletCVConsole project: a secure UI console for Amazon FC teams to view pallet image/video evidence, built with React 18 (React Harmony), Java Lambda (Coral, Smithy, Dagger, CBOR), AWS CDK multi-stage CI/CD, DynamoDB, S3, and Midway SSO + HELIS RBAC.

AZ-1. You designed RBAC with 4+ roles for Amazon FC teams using OAuth, Midway SSO, and HELIS. Walk through the authentication and authorization flow from a user logging in to a query being executed. What does each system do and where is the security boundary enforced?

Answer: Answer: The three systems operate at different layers of the identity stack.

Midway SSO is Amazon’s internal single sign-on system, analogous to an enterprise OIDC/SAML provider. When an FC team member navigates to the console, they are redirected to Midway for authentication. Midway validates their Amazon corporate identity and issues a signed token identifying *who* they are – their Amazon login, org, and badge.

OAuth 2.0 authorization code flow wraps Midway at the application layer. The console registers as an OAuth client; Midway returns an authorization code, the backend exchanges it for a short-lived access token (HttpOnly secure cookie), and every subsequent API call carries that token. OAuth here is the transport mechanism for the Midway identity assertion, not a separate identity store.

HELIS is Amazon’s internal entitlements service – it answers “is this identity permitted to perform this action on this resource?” The backend, after authenticating the token, calls HELIS with the user’s identity and the requested resource (e.g., “view pallet evidence for FC SEA1”). HELIS returns the user’s role (AUDITOR, FC_MANAGER, LOSS_PREVENTION, ADMIN) and the FC scope they are permitted. The backend then enforces that role: AUDITOR can read evidence but cannot delete; LOSS_PREVENTION sees only their assigned FCs.

The security boundary sits at two layers intentionally: (1) the Lambda execution IAM role is scoped to `s3:GetObject` on FC-namespaced prefixes – so even a code bug cannot read another FC’s raw S3 data. (2) The HELIS check gates which FC prefix the application even queries. Defense-in-depth: a bypass in one layer is caught by the other.

AZ-2. Walk through the full image/video evidence retrieval flow in the AFTPalletCVConsole, from the pallet camera system writing to S3 through to the React frontend rendering the asset. Where does the Lambda sit in this path?

Answer: Answer: **Write path:** Pallet CV cameras and PAD (Pallet Anomaly Detection) systems at FCs capture image/video evidence and push it through Amazon’s internal data pipeline to S3. Assets land at a key path organized by FC, date, and evidence type. A metadata record (FC ID, timestamp, pallet ID, asset S3 key, anomaly type) is written to DynamoDB keyed on the pallet/incident ID.

Read path: The React 18 frontend calls the Java Lambda via API Gateway: `GET /api/pallet/{palletId}/evidence`. The Lambda: (1) validates the access token and calls HELIS to confirm the user is permitted for that FC. (2) Queries DynamoDB for the asset S3 key. (3) Calls `S3.generatePresignedUrl(GetObject, bucket, key, TTL=15min)` using the Lambda execution role. (4) Returns the signed URL in the API response. The React component renders `<img src={url}/>` or `<video src={url}/>` – the browser fetches the binary directly from S3. Lambda never touches the binary content.

Why pre-signed URLs instead of Lambda proxying the bytes: proxying image/video bytes through Lambda would consume Lambda memory and execution time per byte transferred, hit API Gateway payload limits (10MB max response), and add cost proportional to data volume. Pre-signed URLs offload the data transfer entirely to S3 – Lambda only produces a URL, and S3 handles all the bandwidth. The URL expires in 15 minutes, limiting the exposure window if it leaks.

AZ-3. What is Pallet CV / PAD (Pallet Anomaly Detection) and what problem does the AFTPalletCVConsole solve for FC teams? Explain the system at the architectural level – what data flows in, what the console surfaces, and who uses it.

Answer: Answer: Amazon FCs handle millions of pallets daily. **Pallet CV** is Amazon’s computer vision system that uses cameras positioned at pallet induction and sort stations to capture images and video of each pallet as it

moves through the FC. **PAD (Pallet Anomaly Detection)** is the ML pipeline that processes that footage to detect anomalies: damaged pallets, incorrectly loaded packages, overheight stacks, or missing items.

The raw evidence (images and video clips associated with each pallet or incident) is stored in S3. Without the console, FC teams (loss prevention, FC managers, auditors) had no unified way to look up evidence for a specific pallet ID or incident – they either had no access to S3 or had to route requests through engineers.

AFTPalletCVConsole is the internal tool that closes this gap. It provides: (1) A search/filter interface where FC staff can query evidence by pallet ID, date range, anomaly type, or FC. (2) In-browser image/video playback via pre-signed S3 URLs. (3) RBAC enforcement so loss prevention at SEA1 can only see SEA1 evidence – not PHX2 or other FCs. (4) Audit logging of who accessed which evidence record, for compliance.

The architectural role of the console is purely a secure access/visualization layer over data that already exists in S3 and DynamoDB – it does not run CV or PAD itself. The Java Lambda + DynamoDB + S3 stack handles the read/query path; the React 18 frontend handles the display; CDK provisions the whole infrastructure.

AZ-4. Your Java backend used CBOR serialization instead of JSON. What is CBOR, why use it for internal service communication, and what are the tradeoffs?

Answer: Answer: **CBOR (Concise Binary Object Representation)** is a binary data serialization format defined by RFC 7049. It encodes the same data model as JSON (maps, arrays, strings, numbers, booleans, null) but as compact binary rather than UTF-8 text.

Why use it for internal Java services: (1) **Payload size:** CBOR typically produces 20–50% smaller payloads than equivalent JSON because it encodes types in a single byte header and avoids quoting strings or writing out field names as repeated UTF-8. For high-throughput internal APIs transferring thousands of evidence records, smaller payloads reduce network I/O and Lambda duration. (2) **Parse speed:** binary formats skip the UTF-8 decode and string-to-number conversion steps that JSON parsers perform. CBOR can be deserialized directly into Java objects faster than Jackson processing equivalent JSON. (3) **Type fidelity:** JSON has no native byte array type – binary fields (like S3 ETags or hash digests) must be base64-encoded in JSON, bloating the payload further. CBOR has a native byte string type.

Tradeoffs: (1) Not human-readable – you cannot `curl` an internal endpoint and read the response directly; debugging requires a CBOR decoder. (2) Tooling: browser clients and external partners expect JSON; CBOR is suitable only for internal service-to-service or service-to-Lambda communication, not the public-facing API Gateway endpoints the React frontend hits. In the console, the Java Lambda used CBOR on internal Coral service calls and JSON on the API Gateway side facing the browser. (3) Schema enforcement is still needed – CBOR’s binary compactness does not add type safety; Smithy’s generated POJOs provide that layer on top.

AZ-5. You used Dagger for dependency injection in the Java Lambda. What is Dagger and why does compile-time DI matter specifically in a Lambda environment compared to a Spring Boot service?

Answer: Answer: **Dagger** is a compile-time dependency injection framework for Java. Unlike Spring’s runtime DI (which uses reflection and classpath scanning to wire beans at application startup), Dagger generates the DI wiring code at compile time – the dependency graph is a set of plain Java classes generated by an annotation processor, not a runtime container.

Why this matters critically for Lambda: Lambda’s performance profile is dominated by cold-start time. The Lambda JVM starts fresh on every cold invocation. Spring Boot’s startup involves classpath scanning (loading all classes, finding `@Component`/`@Bean` annotations, building the `ApplicationContext`) – this typically takes 3–8 seconds for a medium-sized Spring Boot app, which is an unacceptable cold-start for a latency-sensitive internal console.

Dagger generates wiring code that executes at JVM startup as fast as any other Java class initialization – no scanning, no reflection. Cold-start for a Dagger-wired Lambda is measured in milliseconds of initialization overhead, not seconds.

Additional benefits: (1) Compile-time errors: if a dependency is missing or a cycle exists, the build fails – not the Lambda at runtime. Spring surfaces these errors as runtime `NoSuchBeanDefinitionException`. (2) Smaller JAR: Dagger has no framework runtime; the generated code is the dependency graph. Spring Core + Spring Context alone add several MB to the JAR, which also increases cold-start time because Lambda has to download and decompress the package. (3) Testability: Dagger-wired components are plain Java objects; tests inject mock dependencies directly without a Spring test context.

AZ-6. You provisioned the AFTPalletCVConsole with AWS CDK and reduced provisioning time by 70% with a multi-stage, multi-region CI/CD pipeline. Explain how the CDK pipeline is structured – what are the stages, what happens at each stage, and how does promotion work?

Answer: Answer: CDK Pipelines (the `CodePipeline`-backed CDK construct) models deployment as a linear stage sequence. For the console the structure was:

Source stage: triggered on code merge to main. Pulls the repository and runs `cdk synth` to produce the CloudFormation templates. Synth runs unit tests and CDK **Aspects** (policy validators, tagging checks) as a gate – a failed synth blocks all downstream stages.

Beta stage (us-east-1): deploys the full stack (API Gateway, Lambda, DynamoDB table, S3 bucket, CloudFront, IAM roles) to the internal beta environment. Runs integration tests against the deployed stack – a Postman/Newman collection that exercises the HELIS mock, evidence upload, and pre-signed URL generation end-to-end. Approval is automatic if tests pass.

Gamma stage (us-east-1): a production-like environment with real HELIS and Midway integration. Manual approval gate: the tech lead reviews gamma test results before promotion.

Prod stage (multi-region): deploys to the primary region (us-east-1) first, waits for CloudWatch alarms to settle (5-minute bake time), then deploys to the secondary region (us-west-2) as a failover. If the primary region alarm triggers during bake, the pipeline halts and rolls back.

70% provisioning time reduction: the previous process was manual – an engineer ran `aws cloudformation deploy` commands for each resource type in order, referencing a runbook. The CDK pipeline replaced 12 manual steps with one merge. The 70% is the ratio of engineer-hours from manual provisioning to the near-zero engineer-time of automated promotion.

AZ-7. How did you design the DynamoDB table schema for the pallet evidence console? What is the partition key, sort key, and what access patterns did you optimize for?

Answer: Answer: The primary access pattern for FC operations teams is: “show me all evidence for pallet X” and “show me all evidence for FC Y on date Z.” These two patterns drove the key design.

Primary table: partition key = `palletId`, sort key = `timestamp`. This makes the single-pallet lookup $O(1)$: query by `palletId` and scan sort keys in time order to get all evidence for that pallet, newest first using `ScanIndexForward: false`.

GSI for FC + date queries: FC managers need to browse all anomalies for their FC on a given day, not by individual pallet ID. Added a GSI with partition key = `fcId` and sort key = `date#timestamp` (composite: “SEA1#2025-08-14T14:32:00Z”). This collocates all evidence for an FC on the same partition and allows range queries within a date using the `begins_with` condition on the sort key. The React filter UI maps directly to this GSI: selecting FC and date range hits the GSI with `KeyConditionExpression: fcId = :fc AND begins_with(dateTimestamp, :date)`.

Attribute projection: the GSI projects only `palletId`, `timestamp`, `anomalyType`, `thumbnailS3Key` – the fields the list view needs. Full evidence metadata (including all camera angles, resolution, codec) stays in the base table and is only fetched when a user clicks into a specific record. This reduces GSI read cost and response payload for the common browse path.

Avoiding hot partitions: an FC like SEA1 on a peak day could generate thousands of records under one `fcId` partition key. To spread load, a shard suffix (`fcId#shard`) was appended, with the application scatter-gathering across a fixed number of shards and merging results client-side by timestamp.

AZ-8. You built the React 18 frontend (React Harmony) for the console with client-side caching, advanced filtering, and virtual rendering, reducing page load time by 40%. What specifically did you do for each of those three areas?

Answer: Answer: The 40% improvement came from three parallel changes:

1. Client-side caching: The original implementation hit the API on every navigation, even when the user had just viewed the same FC + date range. Added React Query with cache keys [`fcId`, `dateRange`, `evidenceType`]. React Query serves the cached response instantly on navigation and triggers a background refresh only if the data is

older than 60 seconds (stale-while-revalidate). This eliminated the majority of redundant API calls during normal browsing – an FC manager reviewing the same day’s anomalies across multiple tab switches saw zero additional network requests after the first load.

2. API projection + cursor pagination: The original API returned full evidence records in every list response, including fields only needed in the detail view. Added a `fields` query parameter so the list endpoint returns only `id`, `anomalyType`, `timestamp`, `fcId`, `thumbnailUrl`. Payload per page dropped significantly. Added cursor-based pagination (`ExclusiveStartKey` from DynamoDB) rather than offset, so page 10 doesn’t require scanning 9 pages of DynamoDB records first.

3. Virtual rendering: An active FC can generate 1,000+ anomaly records in a day. Rendering 1,000 DOM rows caused a 2-3 second layout paint and sluggish scrolling. Implemented windowed rendering: only the rows currently visible in the viewport exist in the DOM. Scroll events swap rows in and out. Regardless of total result count, the DOM always has at most 30 rows, so initial paint and scroll are instant. The filtering controls (FC dropdown, date picker, anomaly type chips) further reduce the visible set before rendering begins.

AZ-9. You cut Java backend execution time by 35% using low-level design patterns. Beyond CBOR serialization, what specific patterns did you apply and how did you identify the bottlenecks?

Answer: Answer: Identified bottlenecks using Amazon’s internal performance profiling tooling (producing flame graphs). Three hot spots drove the fix:

1. Object pooling for request context: Every API invocation constructed a new `RequestContext` object holding the parsed auth token, FC scope set, and trace ID. Applied object pooling: a fixed pool of pre-allocated `RequestContext` instances is maintained; at request start the handler acquires one from the pool and populates it; at request end it clears and returns it. This eliminated per-request heap allocation and reduced minor GC pause frequency, which was visible as periodic latency spikes in the flame graph.

2. Lazy initialization on evidence record builder: The evidence record builder eagerly populated all metadata fields (camera angle, resolution, codec, GPS bounding box) on every object, even for list-view requests that only needed anomaly type and timestamp. Changed to lazy initialization via Supplier lambdas: each field is only computed when its getter is called. For list-view requests that read 3 of 15 fields, 12 field computations are skipped entirely.

3. Fail-fast predicate ordering in auth check: The authorization predicate chain evaluated 6 conditions left-to-right in an AND. The HELIS network call (15ms) was first. Reordered to cheapest-first: role enum check (nanoseconds), FC scope set lookup (microseconds), then HELIS only if both pass. For the 20% of requests failing the cheap checks (wrong role or out-of-scope FC access attempts), HELIS is never called, saving 15ms per request.

Each change was measured in isolation before combining – flame graph re-run after each fix – to avoid attributing the combined gain to any single change incorrectly.

AZ-10. You used Smithy to define the service model before writing any Java code. Explain what Smithy generates, why model-first design matters at Amazon’s internal service scale, and what you would lose by writing the Java directly.

Answer: Answer: **What Smithy generates:** from a `.smithy` model file defining operations (`GetPalletEvidence(GetPalletEvidenceInput) -> GetPalletEvidenceOutput`), the Smithy build plugin generates: (1) Java interfaces for each service operation. (2) Immutable request/response POJOs with builders, null-safety, and validation. (3) Coral client stubs – any service that wants to call yours imports the generated client; no manual HTTP wiring. (4) Wire protocol bindings for both JSON and CBOR. (5) API documentation.

Why it matters at Amazon’s scale: Smithy is the *source of truth* for the service contract. Any team importing your Smithy model gets a generated client always in sync with the server schema. If you add a required field to `GetPalletEvidenceInput`, every caller sees a compile-time error when they update their client dependency – not a runtime 400 in production. Breaking changes surface at the dependency graph level before any deployment. At Amazon, where an internal service can have dozens of callers across different teams, this eliminates entire categories of API mismatch incidents.

What you’d lose without Smithy: you write Java POJOs and a controller manually. The calling team writes their own POJOs by reading your Javadoc. When a field name changes, they find out from a production error. There is no shared artifact both sides compile against. At two teams this is manageable; at twenty it generates a constant stream of compatibility incidents and backward-compat hacks.

Smithy also enforces **error modeling**: each operation explicitly declares its error shapes. The generated Coral client forces callers to handle those errors at compile time, rather than catching generic exceptions and attempting to parse error message strings – a common source of silent failure in hand-rolled clients.

Part 5: Deep Technical Questions – NYU Projects (10)

NY-1. Your `firebeats_v2.0` code calls the AWS S3 SDK directly from the React Native client with hardcoded access keys in `awsQueries.js`. Why is that dangerous and what is the correct architecture?

Answer: Answer: Embedding long-lived IAM credentials in mobile client code is a critical security anti-pattern for three reasons. First, the app binary can be reverse-engineered – anyone who extracts the APK can grep for the key prefix `AKIA` and immediately obtain credentials that grant `s3:PutObject` with public-read ACL on your bucket. Second, there is no rotation or revocation granularity; if the key leaks you must rotate it globally and redeploy every installed instance. Third, the credentials persist across sessions with no TTL.

The correct architecture has two options depending on the threat model. **Option A (Pre-signed URL via Lambda):** The mobile client POSTs a signed request to an API Gateway + Lambda endpoint; Lambda calls `s3.getSignedUrlPromise('putObject', ...)` with a 15-minute TTL and returns the URL; the client PUTs directly to that URL. Lambda never touches the binary, the credentials live only in the Lambda execution role (IAM instance profile), and the URL expires automatically. **Option B (Cognito Identity Pool):** Configure an Amazon Cognito Identity Pool with an unauthenticated or authenticated role; the mobile SDK calls `Auth.federatedSignIn()` which exchanges the Cognito token for temporary STS credentials (`AssumeRoleWithWebIdentity`) scoped to exactly the S3 operations needed, rotated automatically every hour.

For a production health app handling user data the Lambda pre-signed URL approach is preferable because it lets you add server-side validation (file type, size, user quota) before issuing the URL.

NY-2. Why did you choose Firebase Firestore as the persistence layer for heart rate and health telemetry in `firebeats` instead of a relational database or a time-series store like InfluxDB?

Answer: Answer: The data model is naturally document-oriented and user-partitioned. Each user's heart rate records are stored under `USER_COLLECTION / {userId} / HEART_RATE_SUBCOLLECTION` – a collection-per-user tree that maps cleanly to Firestore's document hierarchy. There are no cross-user JOINS; every query is scoped to a single `userId`, so you never need the relational joins a SQL schema would enable.

Firestore provides three capabilities that were central to the app's design: (1) Real-time `onSnapshot` listeners that push deltas to the client over a persistent WebSocket connection – critical for today's intraday heart rate chart updating as new readings arrive without polling. (2) Offline persistence built into the SDK – the mobile app can cache reads locally so the chart renders even with no connectivity, with automatic sync when the connection returns. (3) Auto-scaling with no schema migrations – the data shape evolved frequently during development (adding vendor-specific keys, subcollection naming changes) and Firestore's schema-free documents meant no ALTER TABLE operations.

The tradeoff is that Firestore cannot do efficient server-side aggregations. Computing a 7-day average heart rate requires reading all daily documents client-side and averaging in JavaScript – exactly what the cache layer does by building the 720-point intraday array. A time-series store like InfluxDB would handle range aggregations more efficiently at scale, but for a research health app with hundreds of users Firestore's operational simplicity and real-time subscriptions outweigh that cost.

NY-3. Walk through the deduplication logic in `heartRateCache.js`. How does the `DATES_FETCHING` map prevent duplicate Firestore reads, and what problem would occur without it?

Answer: Answer: When a user navigates quickly between screen components that each call `getHeartRateData(date, vendor)`, multiple React renders can trigger the same fetch before the first one resolves. Without deduplication, you would fire N parallel Firestore reads for the same date key, each dispatching `SET_HEARTRATE_DATA` on resolution and overwriting state in unpredictable order – a classic read thundering-herd on client state.

The `DATES_FETCHING` map solves this with a promise-keyed deduplication pattern. When a fetch is initiated for key `"YYYY-MM-DD-{vendor}"`, the in-flight Promise is stored in the map. Any subsequent call for the same key checks `DATES_FETCHING.has(key)` and, if true, returns the existing Promise rather than starting a new Firestore read. All callers await the same Promise and receive the same resolved value. Once the Promise settles (or rejects) the key is deleted from the map so future navigations to the same date issue a fresh fetch.

This is structurally equivalent to the **Thundering Herd / Request Coalescing** pattern used at the HTTP layer in

CDNs: collapse concurrent cache misses for the same key into a single upstream request. The implementation is safe because JavaScript is single-threaded – checking and setting the map entry is atomic; there is no race between the `has()` check and the `set()` call.

NY-4. How does the firebeats cache handle today's real-time heart rate differently from historical dates, and what happens when the user changes their wearable device mid-session?

Answer: Answer: Historical dates are static: once fetched from Firestore and stored in `HEART_RATE_DATA_CACHE`, they never change. The cache entry is written once and read on every subsequent navigation to that date – no Firestore listener needed.

Today's date is live. When the app detects `queryDate === today`, it attaches a Firestore `onSnapshot` listener (`hrListener`) on the user's heart rate subcollection for that date. Firestore maintains a persistent WebSocket to the backend; every new heart rate record written to Firestore (e.g., synced from Health Connect by a background service) fires the snapshot callback, which updates `HEART_RATE_DATA_CACHE` for today's key and dispatches a Redux action to re-render the intraday chart. The listener reference is stored module-level so it can be explicitly unsubscribed.

For device-vendor changes, the cache uses composite keys of the form `"YYYY-MM-DD-{vendor}"`. When the user pairs a new wearable, the vendor string changes (e.g., from `"garmin"` to `"fitbit"`), so all previously cached entries for the old vendor are simply never matched again – they become unreachable garbage in the module-level Map. The `removeHrListener()` function, called on component unmount or device-change detection, calls `unsubscribe()` on both `hrListener` and `hrDataListener`, clears both cache Maps, and resets the `DATES_FETCHING` deduplication map, ensuring the next render starts with a cold cache against the new vendor's data.

NY-5. The intraday heart rate chart uses a 720-point array derived from a 1440-element per-minute array. Explain the data transformation and why 720 points instead of 1440.

Answer: Answer: The Health Connect API provides heart rate samples at up to one-per-minute resolution, giving a maximum of 1440 readings for a full day. The cache layer initializes a 1440-element array indexed by minute-of-day (index 0 = midnight, index 1439 = 23:59) and fills each slot with the BPM value from the corresponding Health Connect record.

1440 data points rendered directly on a mobile chart causes two problems: performance (rendering 1440 SVG path vertices strains the React Native charting library's layout engine on mid-range Android devices) and visual noise (adjacent readings one minute apart at similar BPM values produce a jagged, hard-to-read waveform).

The 2-to-1 downsampling fixes both. Pairs of adjacent minutes are averaged: `output[i] = (input[2i] + input[2i+1]) / 2`, producing 720 points representing 2-minute windows. This is a simple **box filter** (mean pooling) – the same operation used in pooling layers of CNNs and in audio downsampling. It smooths transient spikes while preserving the overall BPM trend across the day. The x-axis then maps each index to a 2-minute interval (0, 2, 4 ... 1438 minutes), so the chart correctly labels time of day.

The tradeoff is that brief arrhythmia spikes lasting under 2 minutes can be averaged away. For a fitness monitoring app that is acceptable; a clinical ECG app would need the full 1440-point resolution.

NY-6. In the Broker project, the underwriting service uses `findOrCreate` to upsert records but deliberately does not update the `losses`, `lae`, and `loss_ratio` columns. Why, and what consistency risk does that introduce?

Answer: Answer: The underwriting table has a clear **ownership split**: form-owned fields (`vfbl`, `wc`, `total_premium`, `number_of_claims`) are populated by the broker portal submission flow; carrier-owned fields (`losses`, `lae`, `total_loss_lae`, `loss_ratio`, `points`, `pr_factor`) are set by the insurer's dashboard after actuarial review. The `upsertFromForm` method explicitly comments `"Do NOT update losses, lae..."` in the update path to enforce this boundary.

The rationale is that if a broker re-submits a corrected form (changing the premium figures after a typo), the carrier-owned actuarial data that was already entered should not be clobbered. The form submission and the carrier workflow are two independent write paths operating on different column subsets of the same row.

The consistency risk is a **partial-write race condition**. Sequelize's `findOrCreate` executes a `SELECT` followed by an `INSERT` if not found – this is not atomic in MySQL without a unique index enforcing the constraint at the database level. If two concurrent broker submissions hit `findOrCreate` for the same (`fire_department_id`, `underwriting_year`) pair simultaneously, both `SELECT`s can return not-found before either `INSERT` commits,

leading to a duplicate row or a unique constraint violation. The safe fix is to rely on the database unique constraint and wrap in a try/catch that handles the constraint error and falls back to an UPDATE, or to use a serializable transaction wrapping the SELECT and INSERT. Sequelize's `findOrCreate` does handle this in recent versions using `ON CONFLICT` syntax on PostgreSQL but not on MySQL, which is what Broker uses.

NY-7. Walk through the form extraction pipeline in Broker. A multi-page insurance application arrives as a JSON array of form sections. How does `FormExtractionService` normalize it, and what are the fragility points?

Answer: Answer: The input is an array of objects with shape `[[{formNumber: "1", data: {...fields}}, {formNumber: "2", data: {...}}, ...]`. `FormExtractionService.extractFields()` first flattens all sections into a single object using `Object.assign(flattened, section.data)` – so later sections overwrite earlier ones for duplicate keys.

The flattened object is then queried with multi-alias field lookups. For example, the fire department name tries four field names in order: `entity_name` || `fire_department` || `fire_department_name` || `department_name`. This handles the reality that different insurance forms use inconsistent field naming conventions – form vendors name the same concept differently. The state field defaults to "NY" if absent, which hardcodes geography and would break for non-New-York fire departments.

The fragility points are: (1) **Key collision on Object.assign:** if form 1 and form 2 both include a `county` field with different values (e.g., one is the county of the insured entity, one is a billing county), form 2 silently overwrites form 1. The correct approach would be to namespace by form number: `form1_county` vs. `form2_county`. (2) **Silent null propagation:** if `parseInt("abc")` is called, the method returns `null` and the record is inserted with a null premium – no validation error surfaces to the submitter. (3) **Year extraction heuristic:** `extractUnderwritingYear` parses `effective_date` fields by assuming a date format; any date format variation (MM/DD/YYYY vs. YYYY-MM-DD) can produce the wrong year and silently associate the record with an incorrect policy period.

NY-8. The `react-fuse_19` insurance platform uses both Redux Toolkit and React Query in the same application. What is the intended separation of concerns between them, and where does state management get complicated?

Answer: Answer: The canonical separation is: **React Query** owns **server state** (remote data that lives in the API – fire department records, underwriting summaries, broker profiles) including caching, background refetching, stale-while-revalidate, and optimistic updates. **Redux Toolkit** owns **client/UI state** (user session, auth tokens, modal open/close, sidebar filters, selected rows in a table, Stripe checkout step) – state that has no server counterpart and needs to be shared across component trees without prop drilling.

The complication arises in the authentication flow. AWS Amplify Auth stores the Cognito session in `localStorage` and exposes it asynchronously via `Auth.currentAuthenticatedUser()`. Bridging Amplify's async auth state into Redux requires a thunk or middleware that calls Amplify on startup and dispatches a `setUser` action. React Query's headers then need the token from Redux state to attach to API calls – creating a dependency where server-state (React Query) consumes client-state (Redux) to construct requests. If the token is stale or the Amplify session expires mid-session, React Query's automatic background refetch will fire with an expired token, the API will return 401, and the query enters error state. The correct pattern is to configure React Query's `retry` callback to not retry on 401 and instead dispatch a Redux `logout` action that clears state and redirects to the Amplify hosted UI.

A second complication: Stripe billing status is fetched via React Query but gates feature visibility in the UI (RBAC-by-plan). Rendering decisions that depend on both the Stripe subscription status (server state) and the user's RBAC role (stored in Redux) require components to consume both stores simultaneously, making the component harder to test and the state harder to trace.

NY-9. Broker uses AWS Amplify to host its Lambda backend. What does that architecture look like end-to-end, and what are the operational tradeoffs compared to deploying your own API Gateway + Lambda stack as you did in `CampusMesh`?

Answer: Answer: Amplify's `amplify/backend/function/brokerapi` structure generates a CloudFormation stack that provisions: an API Gateway REST API, a Node.js Lambda function (the Express/Sequelize app under `src/`), an IAM execution role, and log groups. `amplify push` compiles the stack and deploys it. The frontend (`react-fuse_19`) uses the Amplify JS library which reads a generated `aws-exports.js` config file with the API endpoint, Cognito pool IDs, and S3 bucket names – no manual environment variable wiring.

The operational advantages for a small team are: zero infrastructure boilerplate, built-in Cognito Auth integration, and a single `amplify push` deploys both backend and frontend config atomically. For a research project with a small user base these advantages dominate.

The tradeoffs versus the CampusMesh approach (explicit Serverless Framework + `serverless.yml`): (1) **Visibility**: Amplify’s generated CloudFormation is opaque – if something fails during `amplify push`, the error messages are often in CloudFormation event logs several screens away. CampusMesh’s `serverless.yml` makes every IAM policy, SSM reference, and function config explicit and version-controlled. (2) **Environment management**: Amplify environments (`amplify env add`) are less flexible than the Serverless Framework’s `-stage` parameter; multi-environment promotion in Amplify requires careful `amplify env checkout`. (3) **Secret management**: CampusMesh uses SSM Parameter Store with KMS encryption and least-privilege IAM per function; Amplify injects environment variables directly into the Lambda configuration, which exposes them in the AWS console in plaintext.

NY-10. In the Broker project, WebSocket-based agent messaging is used for real-time chat between brokers and agents. How is the WebSocket connection managed in an Amplify Lambda backend, and what is the fundamental scaling limitation?

Answer: Answer: The Broker backend’s `agentmessages.model.js` and `agentform.controller.js` support a messaging flow where brokers and agents exchange messages on insurance applications. In an Amplify-hosted Lambda, the connection lifecycle is: the client opens a WebSocket (likely via API Gateway WebSocket API or via a polling/long-poll mechanism over REST, since Amplify’s default API category is REST), the Lambda function handles the frame, writes to the `AgentMessages` Sequelize model (MySQL), and broadcasts to connected clients.

The fundamental scaling limitation is the same one that exists in CampusMesh’s `websocketHandler.js`: **Lambda is stateless and ephemeral**. Any in-memory connection registry (a Map or array of socket objects) only exists within a single Lambda container. When API Gateway routes WebSocket frames across multiple Lambda instances (which happens as concurrency scales), a message sent by broker A hits instance 1 but agent B’s connection is registered on instance 2 – the broadcast never reaches the recipient.

The correct solution for both projects is to externalize connection state: store active `connectionId` values in DynamoDB with a TTL equal to the maximum WebSocket idle timeout. When a `MESSAGE_SENT` event arrives, the handler queries DynamoDB for all `connectionIds` in that conversation, then calls `ApiGatewayManagementApi.postToConnection(connectionId, payload)` for each one. This makes every Lambda instance stateless and the connection registry durable, at the cost of an additional DynamoDB read per broadcast. For the Broker use case, where a conversation typically has two participants (one broker, one agent), this adds one DynamoDB query per message – negligible overhead.

Part 6: Docker and Kubernetes (5)

Questions grounded in container and orchestration concepts relevant to the Nordstrom role – covering image layering, container networking, Kubernetes scheduling, health checks, and rolling deployments.

DK-1. Explain the Docker image layer model. How does layer caching work during a build, and what is the practical consequence of instruction ordering in a Dockerfile?

Answer: Answer: A Docker image is a stack of read-only layers. Each instruction in a Dockerfile (FROM, RUN, COPY, ADD) produces one layer – a diff of filesystem changes from the previous layer. When you run a container, Docker adds a thin writable layer on top; the image layers themselves never change.

Layer caching: the Docker build daemon caches each layer keyed by the instruction text and the content of any files copied. When you rebuild, Docker walks instructions top-to-bottom and reuses cached layers until it hits the first cache miss – then it must rebuild that layer and every layer below it. This is the critical consequence of instruction ordering.

Practical example: if your Dockerfile copies application source code before installing dependencies:

```
COPY . /app
RUN npm install
```

Every time any source file changes, the COPY layer is a cache miss, which forces `npm install` to re-run (slow). The correct order places dependency installation before source copy:

```
COPY package.json package-lock.json /app/
RUN npm install
COPY . /app
```

Now `npm install` is only re-run when `package.json` changes – the common case of only changing source code hits the cached install layer and builds in seconds.

The same principle applies to Java (copy `pom.xml/build.gradle` and run dependency download before copying `src/`) and Python (copy `requirements.txt` and run `pip install` before copying application code).

DK-2. What is the difference between a Docker bridge network and host networking, and when would you use each? How do containers on the same bridge network discover each other?

Answer: Answer: **Bridge network** (default): Docker creates a virtual network interface (`docker0`) on the host. Each container gets its own network namespace with a private IP (e.g., `172.17.0.x`). Traffic between containers is routed through the virtual bridge. Containers are isolated from the host network – a process inside the container listening on port 8080 is not accessible from the host unless you explicitly publish the port (`-p 3000:8080` maps host port 3000 to container port 8080).

Service discovery on a user-defined bridge: on Docker’s default bridge, containers cannot resolve each other by name. On a *user-defined bridge* (`docker network create mynet`), Docker’s embedded DNS resolver automatically maps container names to their IPs. A container named `api` is reachable from another container on the same network simply as `http://api:8080` – no IP lookups needed. This is how **docker-compose** service networking works: compose creates a user-defined bridge and names containers after their service names.

Host network (`-network host`): the container shares the host’s network namespace. A process listening on port 8080 inside the container is immediately accessible on host port 8080 without any port mapping. No NAT overhead. Use cases: high-performance workloads where bridge NAT latency is measurable, or tools that need to inspect host network interfaces (monitoring agents, packet capturers).

Tradeoff: host networking removes isolation – a misconfigured container can bind ports that conflict with host services. Bridge networking is the right default for application services; host networking is reserved for infrastructure tooling.

DK-3. In Kubernetes, what is the difference between a liveness probe and a readiness probe? What happens when each one fails, and what is the consequence of misconfiguring them?

Answer: Answer: **Liveness probe:** answers “is this container alive or deadlocked?” If the liveness probe fails

(after `failureThreshold` consecutive failures), kubelet kills the container and restarts it according to the pod's `restartPolicy`. Use case: detecting a container that has entered an unrecoverable state – a deadlock, an OOM that didn't kill the process, a corrupted in-memory state – where restarting is the correct recovery action.

Readiness probe: answers “is this container ready to receive traffic?” If the readiness probe fails, the pod is removed from the endpoints list of all matching Services – load balancers and kube-proxy stop routing traffic to it. The container keeps running; it is not restarted. Traffic resumes automatically when the readiness probe passes again. Use case: a container that is starting up (JVM warmup, Spring context loading, caches being primed) or temporarily busy (processing a large job, dependency unavailable) should not receive traffic during that window.

Misconfiguration consequences:

- **Liveness probe too aggressive** (e.g., short `initialDelaySeconds`, short timeout): kills a healthy container during slow startup. The container restarts in a loop (`CrashLoopBackOff`). A Node.js or JVM app that takes 30s to start should have `initialDelaySeconds: 45` to avoid this.
- **No readiness probe:** pods receive traffic before they are ready, causing request failures during deployments and after pod restarts.
- **Readiness probe pointing at a deep dependency** (e.g., the probe calls an endpoint that queries the database): if the database is slow, all pods fail readiness simultaneously, dropping 100% of traffic. Readiness probes should check the application's own health, not external dependencies.

DK-4. How does a Kubernetes rolling deployment work? What do `maxSurge` and `maxUnavailable` control, and how do you roll back if the new version is bad?

Answer: Answer: A rolling deployment replaces pods incrementally rather than all at once. The Deployment controller maintains two `ReplicaSets` – the old version and the new version – and gradually scales the new one up while scaling the old one down.

maxSurge (default 25%): the maximum number of pods that can exist above the desired replica count during the rollout. With 10 replicas and `maxSurge=2`, you can have up to 12 pods running at once – 10 old + 2 new. Controls how fast the new version spins up; higher `maxSurge` = faster rollout but more resource consumption during the transition.

maxUnavailable (default 25%): the maximum number of pods that can be unavailable (below desired count) during the rollout. With 10 replicas and `maxUnavailable=1`, at least 9 pods must be passing readiness at all times. Controls the minimum traffic capacity during rollout; set to 0 for zero-downtime deployments (requires `maxSurge > 0` to make progress).

Rollback: `kubectl rollout undo deployment/myapp` reverts to the previous `ReplicaSet` instantly – Kubernetes scales up the old RS and scales down the new RS, following the same `maxSurge`/`maxUnavailable` constraints. `kubectl rollout history deployment/myapp` lists revision history; `kubectl rollout undo --to-revision=N` targets a specific prior version. The rollback speed is the same as the rollout speed – not instant, but controlled.

Detecting a bad rollout automatically: configure a readiness probe on the new pods. If new pods fail readiness, the Deployment controller stalls the rollout – it will not scale down the old pods further until the new ones are ready. This means a bad deployment automatically stops progressing rather than taking down all capacity.

DK-5. Your NYU insurance platform was deployed via Docker on EC2. What are the operational differences between that approach and running the same containers on Kubernetes? When is the simpler EC2+Docker setup the right choice?

Answer: Answer: **EC2 + Docker (the NYU approach):** you SSH into one or more EC2 instances, pull the image, and run `docker run` (or `docker-compose up`). Scaling means manually launching more EC2 instances and running the same command. There is no automated health monitoring – if the container crashes, you write a `systemd` unit or a shell restart loop. Deploys are a manual pull-and-restart or a simple bash script.

Kubernetes: the control plane handles scheduling (which node to place a pod on based on resource requests), health monitoring (liveness/readiness probes with automatic restart), scaling (HPA adjusts replica count based on CPU/memory metrics), and rolling deployments. You describe desired state in YAML; Kubernetes continuously reconciles actual state to match.

What Kubernetes adds over raw Docker:

- **Self-healing:** crashed pods are restarted automatically; containers on failed nodes are rescheduled elsewhere.
- **Service discovery + load balancing:** a Service object gives pods a stable DNS name and VIP; kube-proxy load-balances across pod IPs transparently.
- **Config and secret management:** ConfigMaps and Secrets decouple runtime config from images; changes can be applied without rebuilding.
- **Resource isolation:** CPU and memory requests/limits prevent one container from starving another on the same node.

When EC2+Docker is the right choice: for a research platform with 6 clients and predictable low traffic, Kubernetes adds significant operational overhead (cluster management, YAML complexity, learning curve) without proportional benefit. Docker on EC2 is simpler to reason about, cheaper to operate, and faster to deploy for a small team. Kubernetes makes sense when you need multi-service coordination, auto-scaling under variable load, or multi-node resilience – typically at >10 services or production SLA requirements.

Quick Reference: Key Numbers to Remember

Metric	Value
Amazon: Backend execution time reduction (Java low-level patterns)	35%
Amazon: Page load time reduction (React 18 caching + virtual render)	40%
Amazon: CI/CD provisioning time reduction (CDK pipeline)	70%
CampusMesh: Environments deployed	4 (local, dev, staging, prod)
CampusMesh: IAM roles in access control	4+ roles
CampusMesh: CampusIQ accuracy improvement (RAG)	30%
NYU / InviLite: RAG accuracy improvement	56% → 92%
NYU / InviLite: Incident resolution time	1–2 hours → minutes
NYU ETL: Records processed	250K+
NYU Insurance Platform: Revenue generated	\$50K+ across 6 clients
NYU Health Platform: NSF funding	\$100K
PwC: OCR processing time reduction	75%
PwC: Batch ingestion time reduction	5 hours → 1 hour
PwC: Batch record size	100K rows / day
PwC: Production defect reduction	25%
PwC: Reporting efficiency improvement (Power BI)	40%

Day-of Checklist

- **Opening intro (30 seconds):** “I am finishing my MS in Computer Engineering at NYU. Last summer I was a SDE intern at Amazon building a secure internal data console for fulfillment center teams – React 18 frontend, Java Lambda backend with Coral and Smithy, and AWS CDK infrastructure. Currently I am at CampusMesh building Node.js microservices on a serverless AWS backend. I am excited about Nordstrom specifically because the Customer Experience Platform means direct, measurable impact on how real customers shop.”
- **For every technical answer:** Anchor to a specific project and number. Do not answer in the abstract.
- **For every behavioral answer:** STAR format. Situation + Task in max 2 sentences. Most of the time on Action (your specific choices, not the team’s). Close with a quantified result.
- **Nordstrom values to weave in:**
 - Customer Obsessed: user feedback changed your CampusIQ design (B2, B17)
 - Owners at Heart: picked up CDK work at Amazon that was not yours (B1, B6)
 - Curious & Ever Changing: adopted FAISS/Bedrock instead of Elasticsearch (B13)
 - Here to Win: Amazon deadline management (B11), PwC sprint rescue (B4)
 - We Extend Ourselves: PwC mentoring and code review checklist (B5, B16)
- **Questions to ask Sangeeta Nori:**
 1. “What does the Customer Experience Platform’s critical path look like technically – what services are in play when a customer completes a purchase?”
 2. “What is the team’s biggest engineering challenge right now?”
 3. “How does the team currently think about AI-assisted development – is there a shared framework for when to use it and how to validate it?”
 4. “What does success look like for an Engineer 1 at the 6-month mark?”
- **If you don’t know the answer:** Say “Let me think through that” and reason out loud. Senior managers value how you think, not just what you already know.